# Week 5: Suricata IDS on pfSense, and the "Filtered" Mystery

**Short version:** I installed Suricata, an intrusion detection system (IDS), on my pfSense firewall. The install looked broken, which led me into upgrading the firewall's whole operating system. Once Suricata ran, my Nmap scans showed every port as "filtered", and it took me weeks and a proper step-by-step investigation to find out why. The cause was not the firewall. It was a wrong route on my Kali machine.

This writeup tells the story in the order it happened, explains every idea in plain words, and keeps the evidence (screenshots) next to each claim.

---

## Contents


1. What I was trying to build
2. Part 1: The install that looked broken
3. Part 2: Upgrading pfSense itself
4. Part 3: A clean Suricata install
5. Part 4: The scans that said "filtered"
6. Part 5: Finding the real cause
7. Part 6: Real alerts and my own rule
8. What I learned
9. Cheat sheet
10. Still open


---

## 1. What I was trying to build

**The goal:** put a "security camera" on my network that watches traffic and raises an alert when something looks like an attack.

**The lab (all virtual machines, fully isolated from the internet):**

```
  Kali (attacker)                    pfSense (firewall + Suricata)
Metasploitable2 (target)
 192.168.58.10  ----eth1----  LAN em0 192.168.58.1 | WAN em1 192.168.59.1  ----  192.168.59.20
                                                ^
                                   Suricata watches the WAN side
                                   (OPT1 em2 = NAT, internet for package downloads only)
```

**Plain words:** - **Kali** is the "attacker" machine I run tools from. - **pfSense** is the firewall in the middle. Every packet between the two sides passes through it. - **Metasploitable2** is a server that is deliberately full of security holes, so I can practise on something legal and safe. - **Suricata** is the camera. It compares traffic against a library of known attack patterns (called *rules*) and logs an alert on a match.

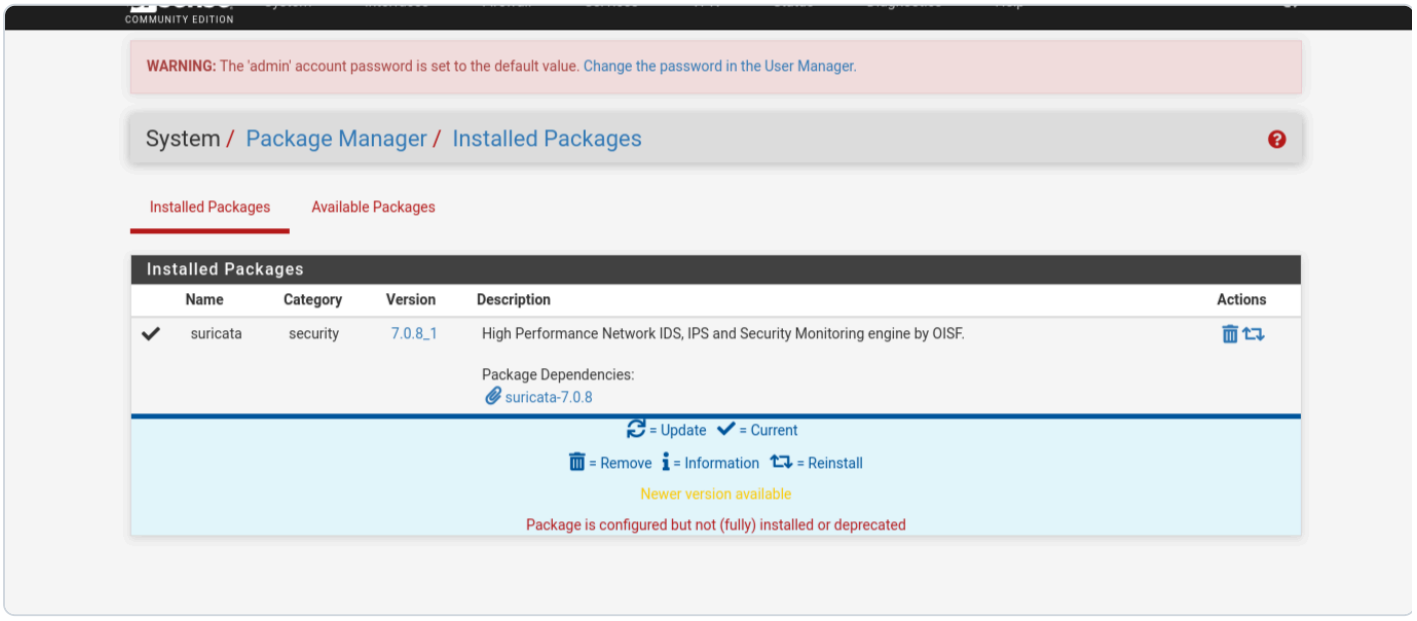
**IDS vs IPS:** an IDS only watches and reports. An IPS also blocks. My Suricata is set to **Blocking Mode: DISABLED**, so it is an IDS.

---

## Part 1: The install that looked broken

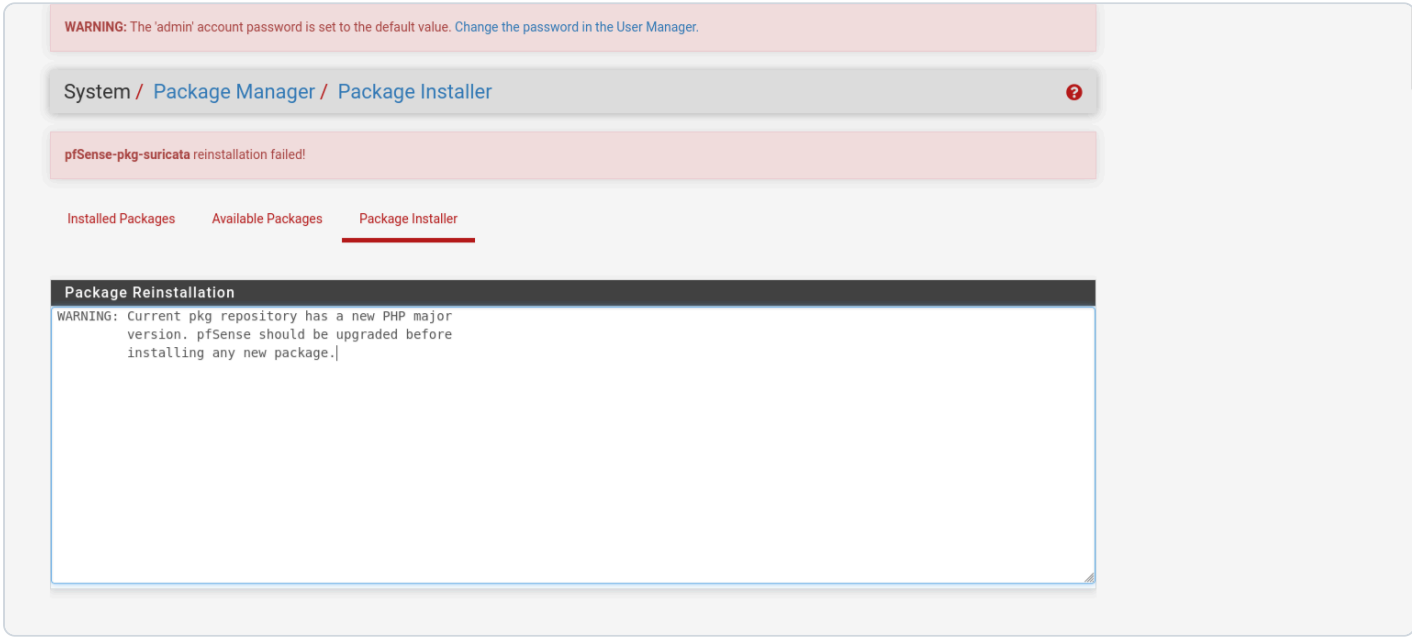
I installed the Suricata package from pfSense's Package Manager. Then the *Installed Packages* page showed:

> Package is configured but not (fully) installed or deprecated




My first thought was "Suricata is broken, reinstall it". The reinstall failed with a much more helpful message:

> WARNING: Current pkg repository has a new PHP major version. pfSense should be upgraded before installing any new package.



### Explain it like I'm five

Imagine pfSense is a phone and Suricata is an app. The app store has moved on and only sells apps built for the **newest** version of the phone's system. My phone was on an **old** version (pfSense 2.7.2), and the store expected 2.8.1. The app was not broken. My phone was too old for it.

### Why this matters beyond one lab

This is a common pattern. `apt install` fails because a library is too old, `pip install` fails because Python is too old, a phone app refuses to install on an old Android. A package manager does not just copy files. It first checks "does this match what you are running?" When an install fails, the problem is often underneath the thing you are installing.

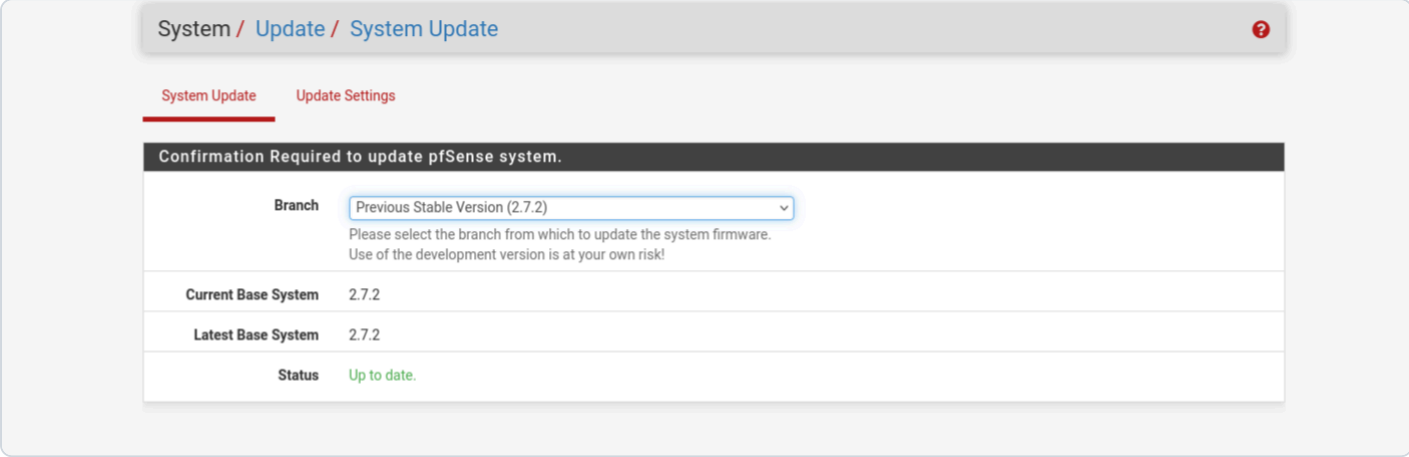
---

## Part 2: Upgrading pfSense itself

I had to move the base system from **2.7.2 to 2.8.1**. That turned into four separate problems.

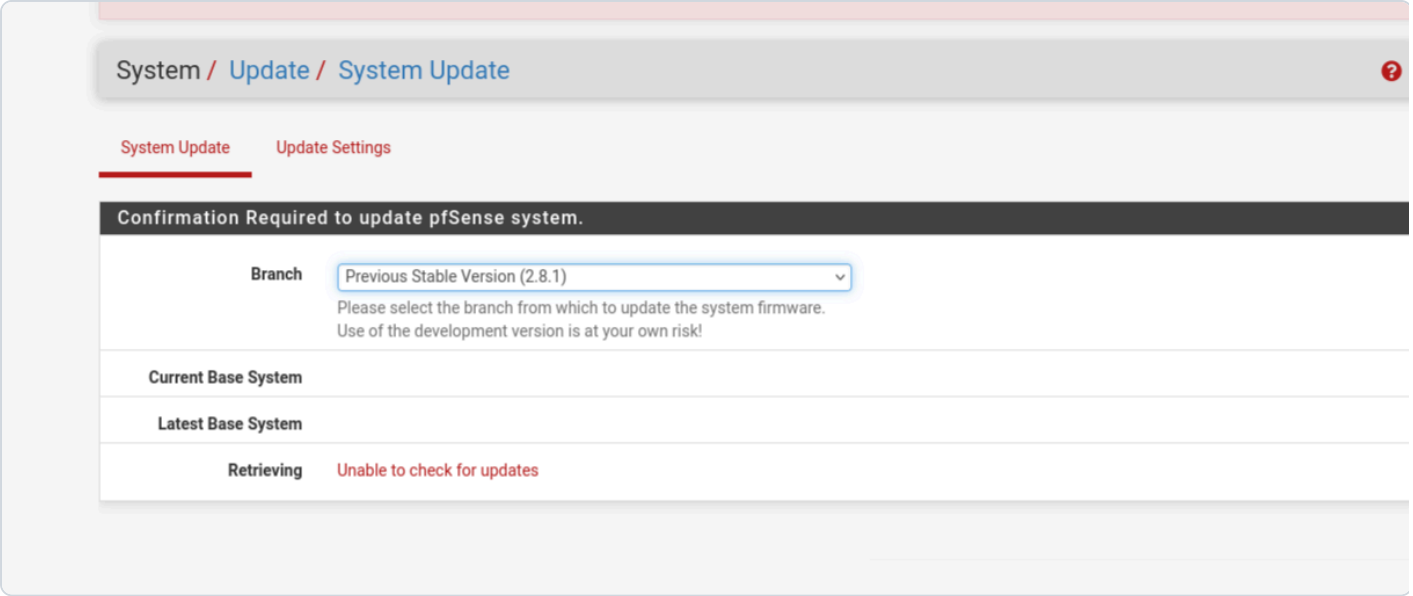
### Problem 1: "Up to date" was wrong

The System Update page said I was up to date. It was comparing against the wrong track: the **Branch** dropdown was set to 2.7.2, so of course 2.7.2 was the latest.




*Simple version:* it is like asking "is there a new season?" while the app is pointed at Season 1's page.

I switched the branch to 2.8.1. The page then said **Unable to check for updates**.

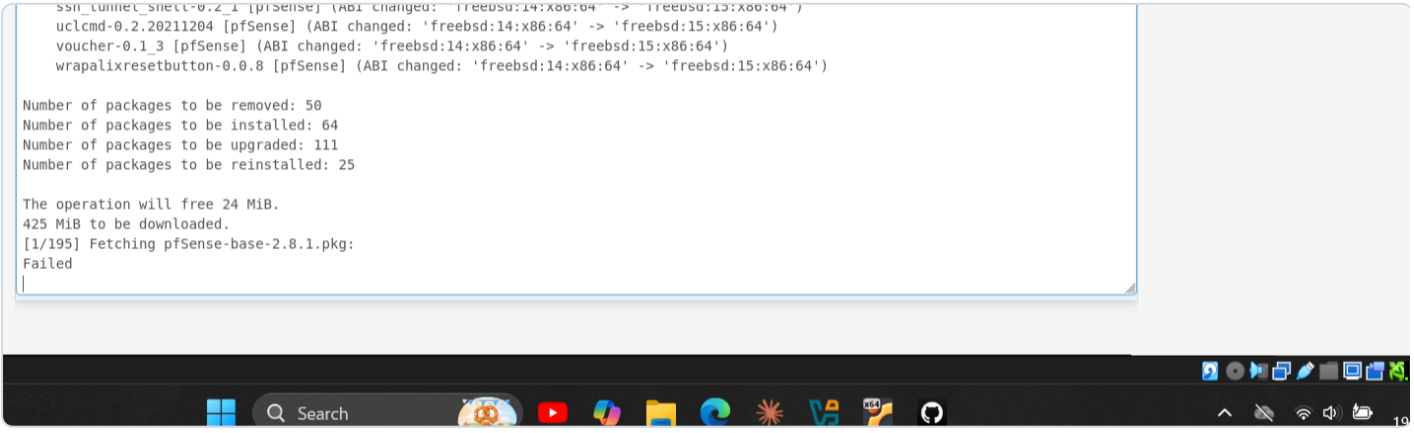



## Problem 2: The download died halfway

When the update finally started, it listed a huge job: **425 MiB** to download and over 190 packages to change. Every package showed `ABI changed: 'freebsd:14' -> 'freebsd:15'`. The **ABI** (Application Binary Interface) is the low-level contract between programs and the operating system. A change from `freebsd:14` to `freebsd:15` means the whole foundation was being swapped.

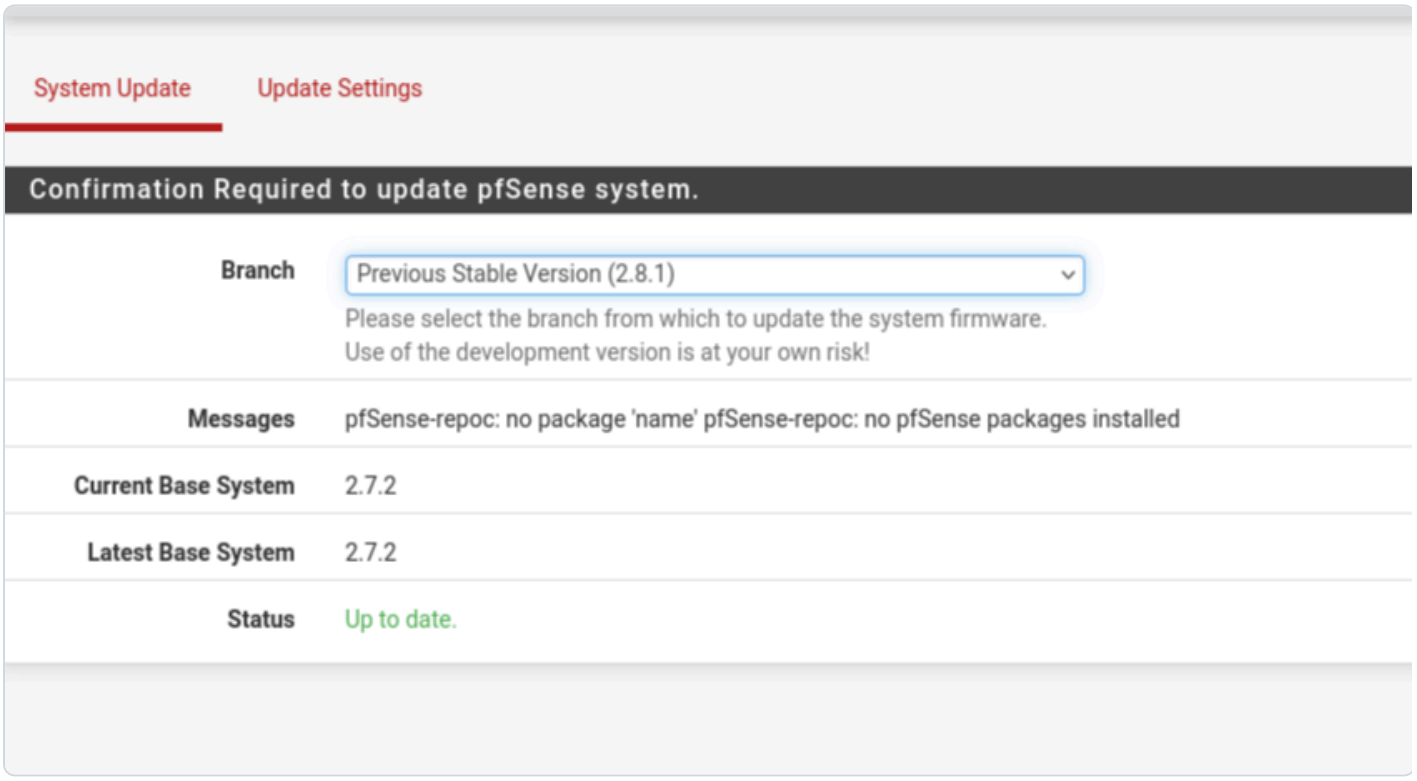



The very first file failed to download. My internet connection was not stable enough for one big transfer.



### Problem 3: The failure left a broken catalog

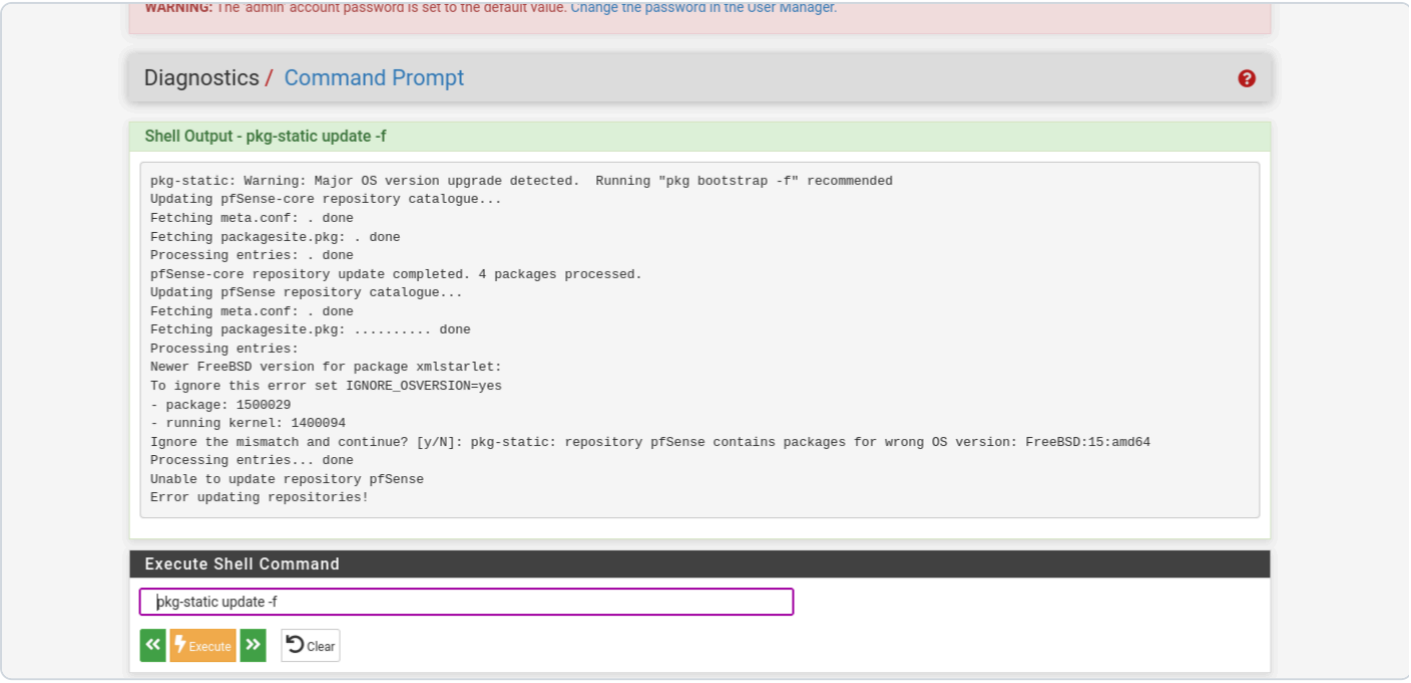
After the failure, pfSense could no longer read its list of available packages. The update page showed confusing messages:




I used pfSense's built-in **Diagnostics > Command Prompt** to force a fresh catalog download:

```
pkg-static update -f
```

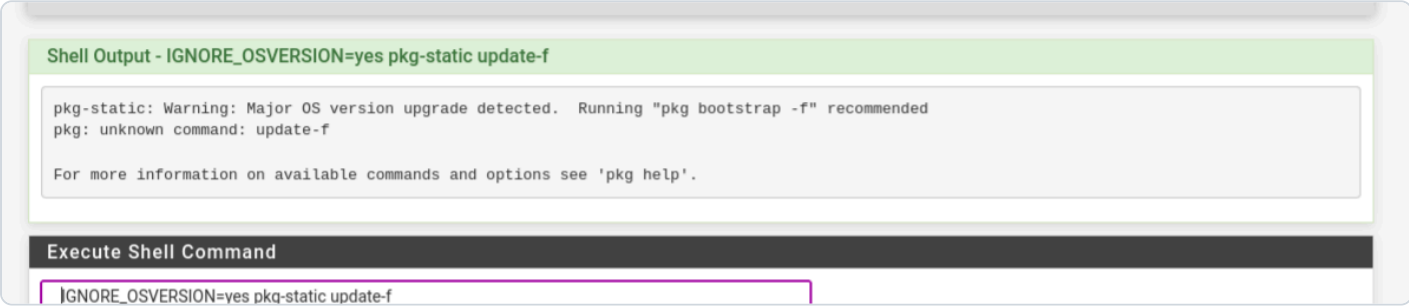
`-f` means *force*: throw away the old catalog and fetch a new one. pkg then asked a yes/no question about the OS mismatch, which a web command box cannot answer:




The fix is to answer "yes" in advance with an environment variable:

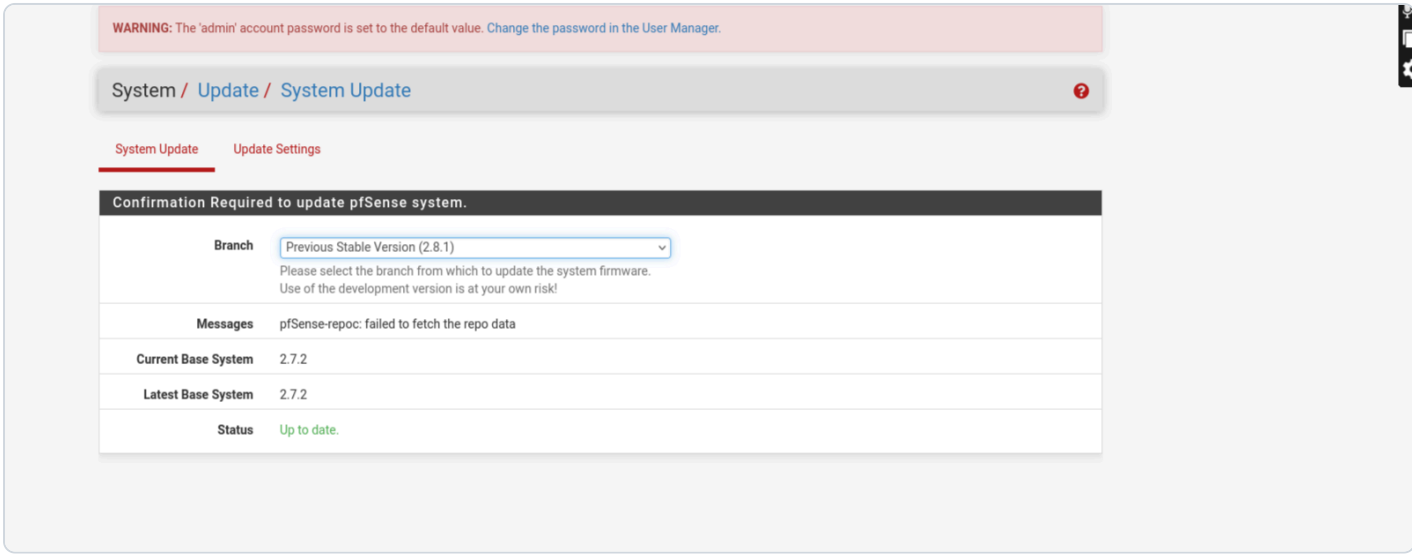
```
IGNORE_OSVERSION=yes pkg-static update -f
```

My first try had a typo ( `update-f` with no space), so pkg said `unknown command` :



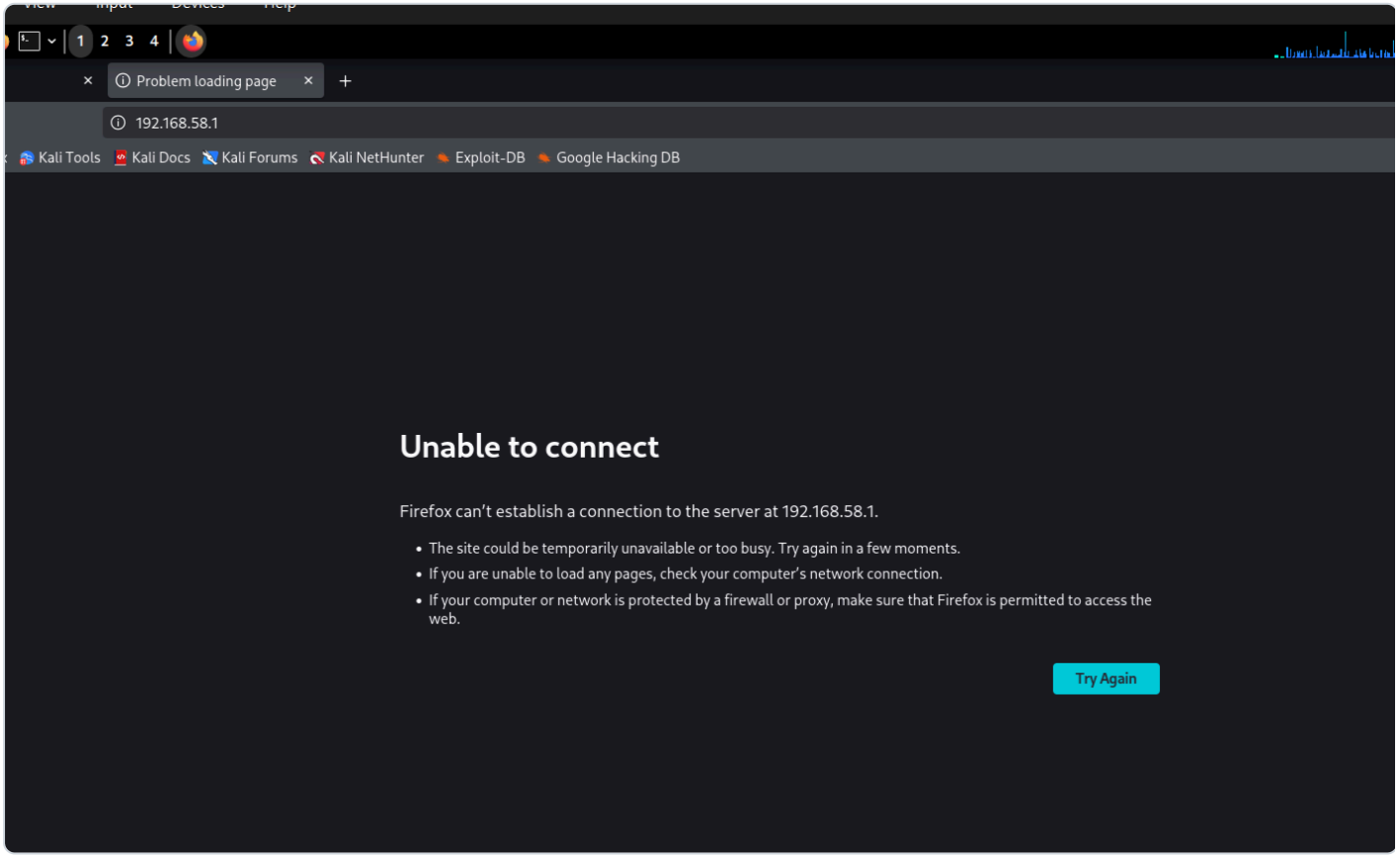

*Lesson: one missing space can look like a deep technical failure. Copy and paste commands.*

I also tried `pfSense-repoc` , which is a separate tool from `pkg-static` , and learned it does not accept the same options. The update page still reported `failed to fetch the repo data` :

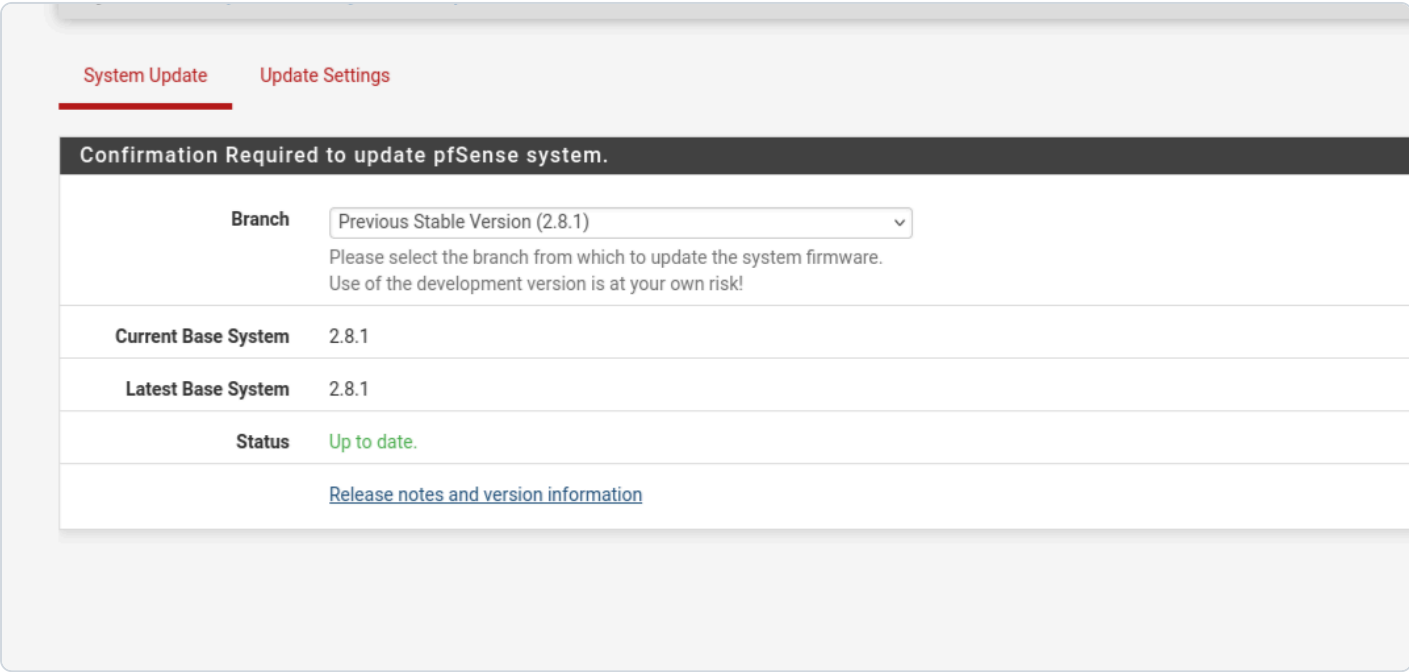



## Problem 4: The update, at last

With the catalog fixed and a steadier connection, I retried the update. pfSense rebooted. For a few minutes the web interface was unreachable, which is expected during a reboot:



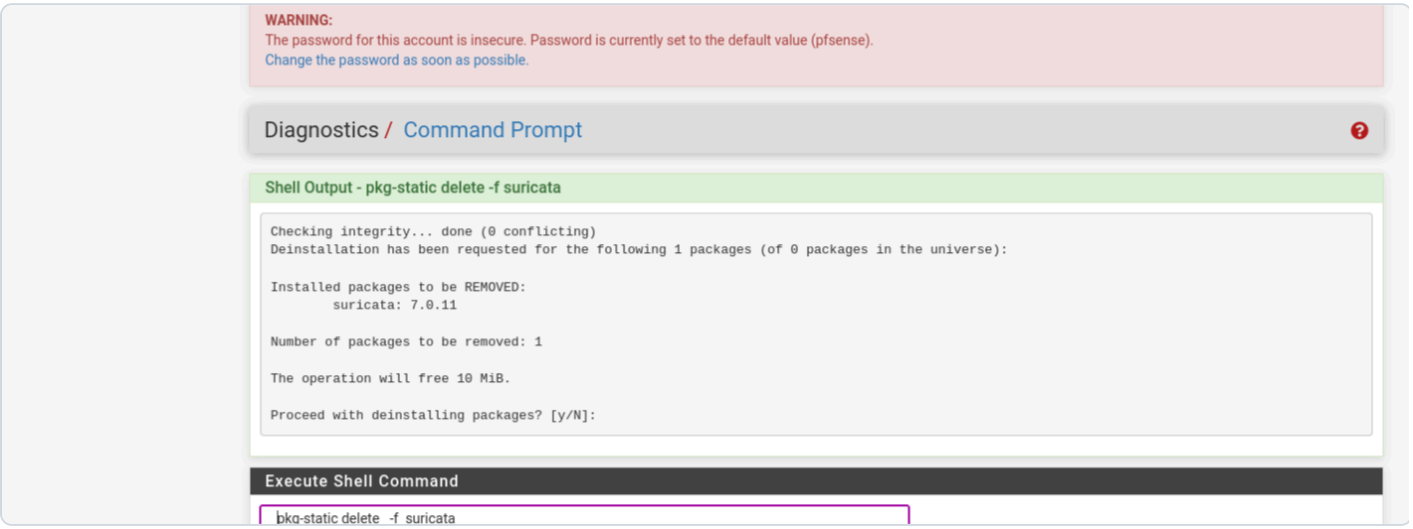

When it came back, the page confirmed the real result:




## Part 3: A clean Suricata install

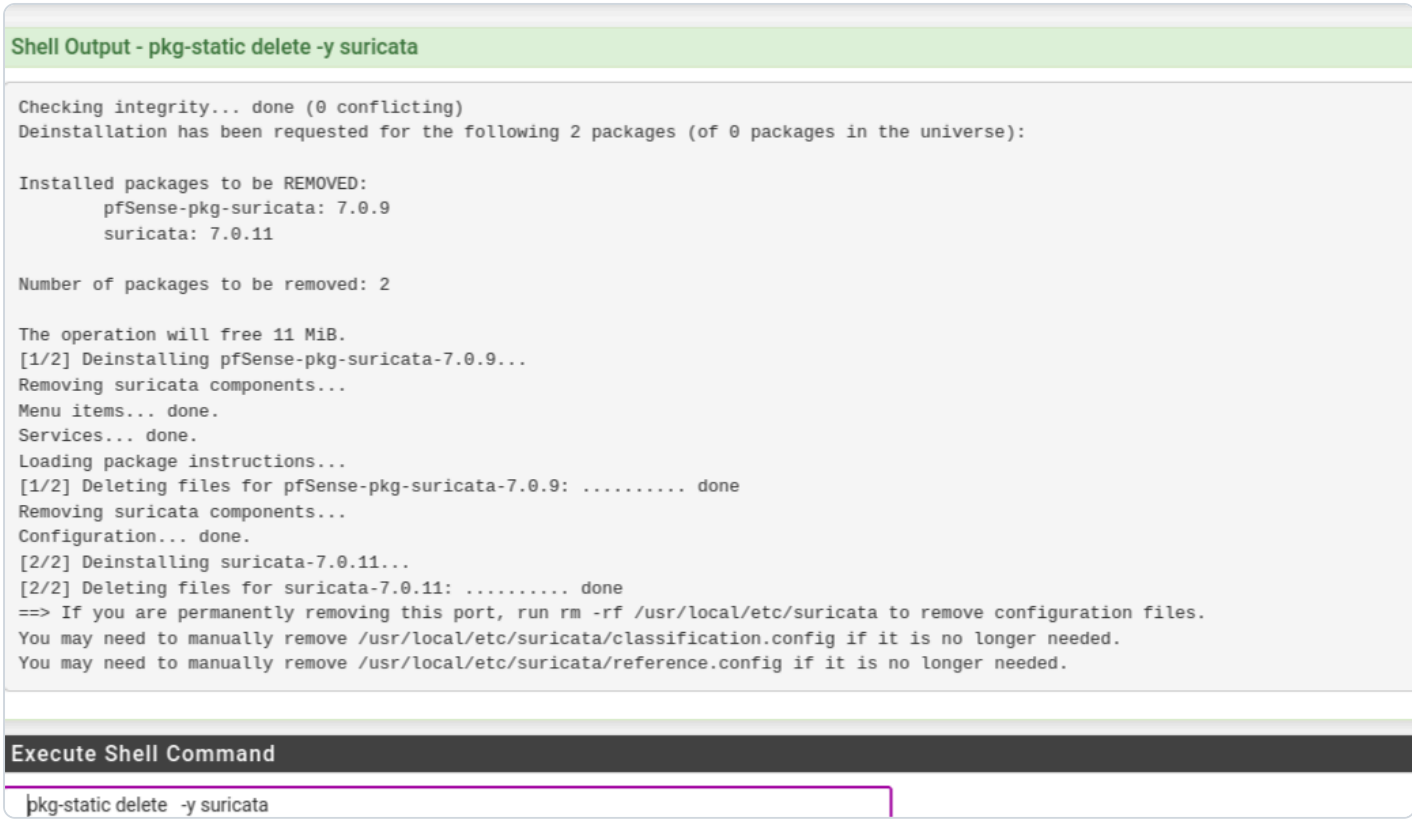
The old Suricata was tied to the old operating system, so I removed it completely and started fresh.

The delete command stopped at a yes/no prompt that the web command box cannot answer:



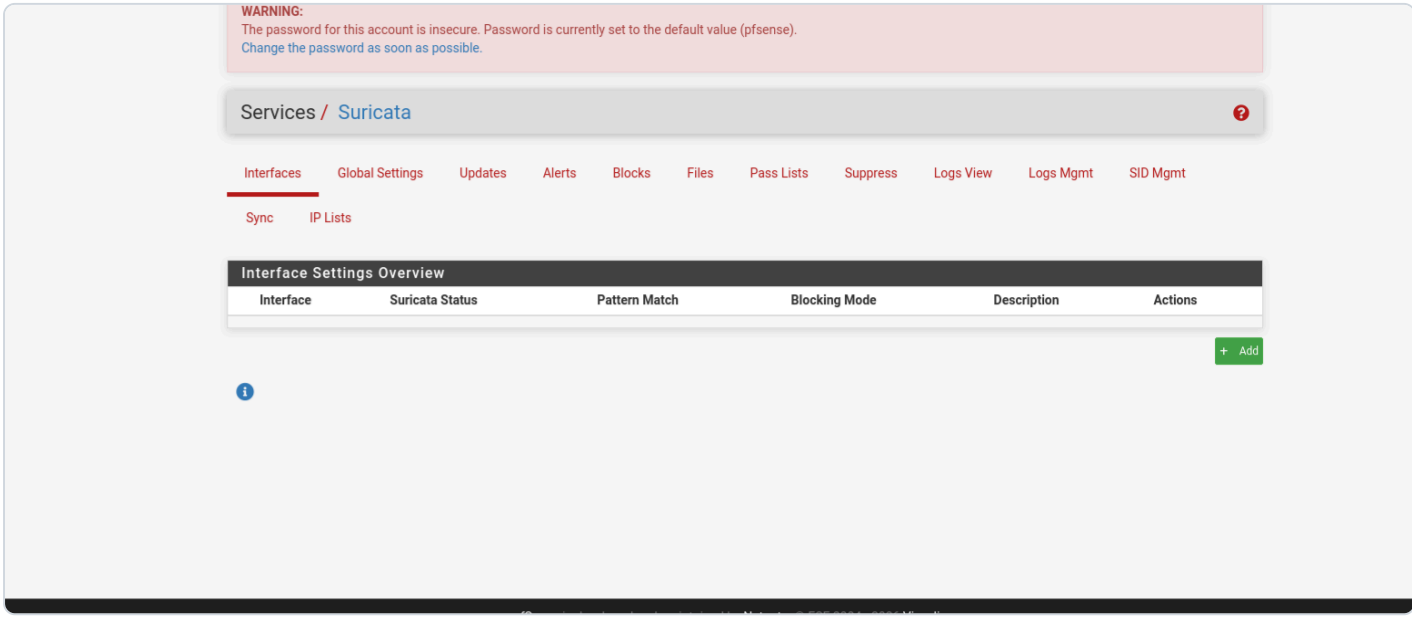

The fix is the `-y` flag, which means "assume yes":

```
pkg-static delete -y suricata pfSense-pkg-suricata
```

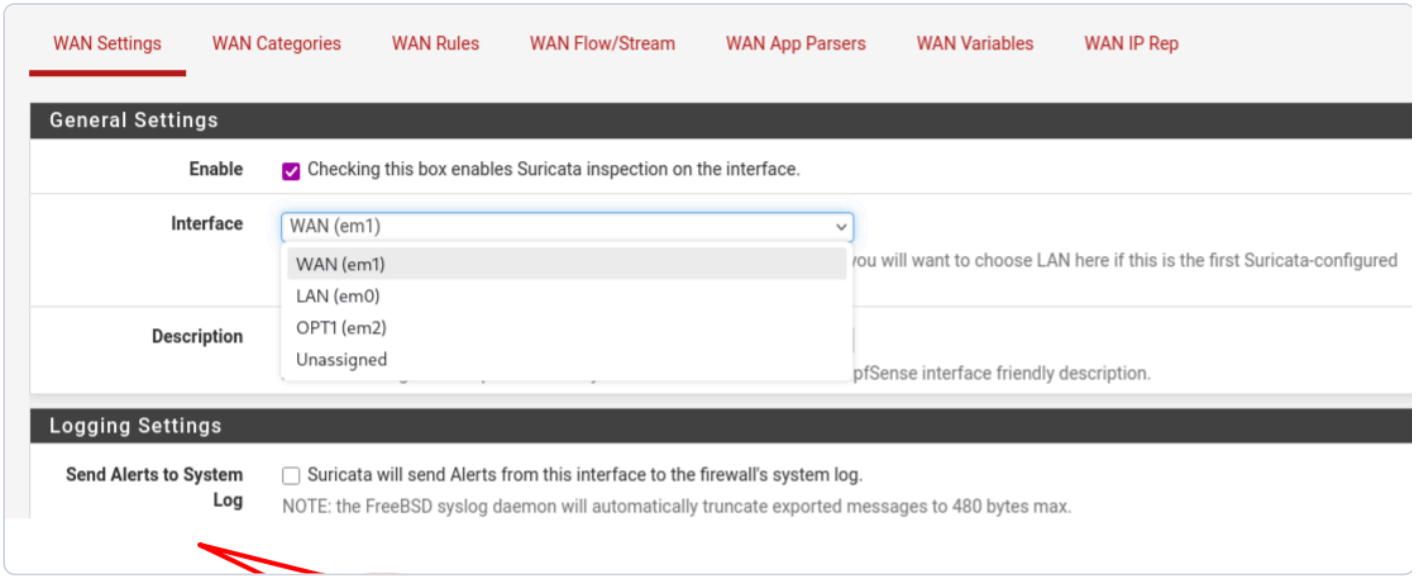


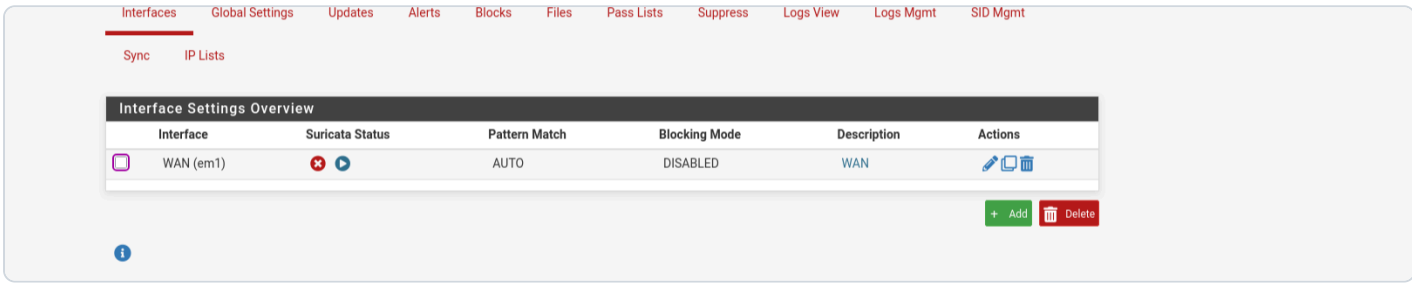

*Remember: web-based command boxes cannot answer interactive prompts. Look for a `-y` or similar flag.*

I reinstalled both packages from the Package Manager. The "configured but not fully installed" banner **stayed on the page**, but I stopped trusting it. The real test is whether the feature works, so I opened **Services > Suricata**. The menu was complete and working, just with no interfaces yet:

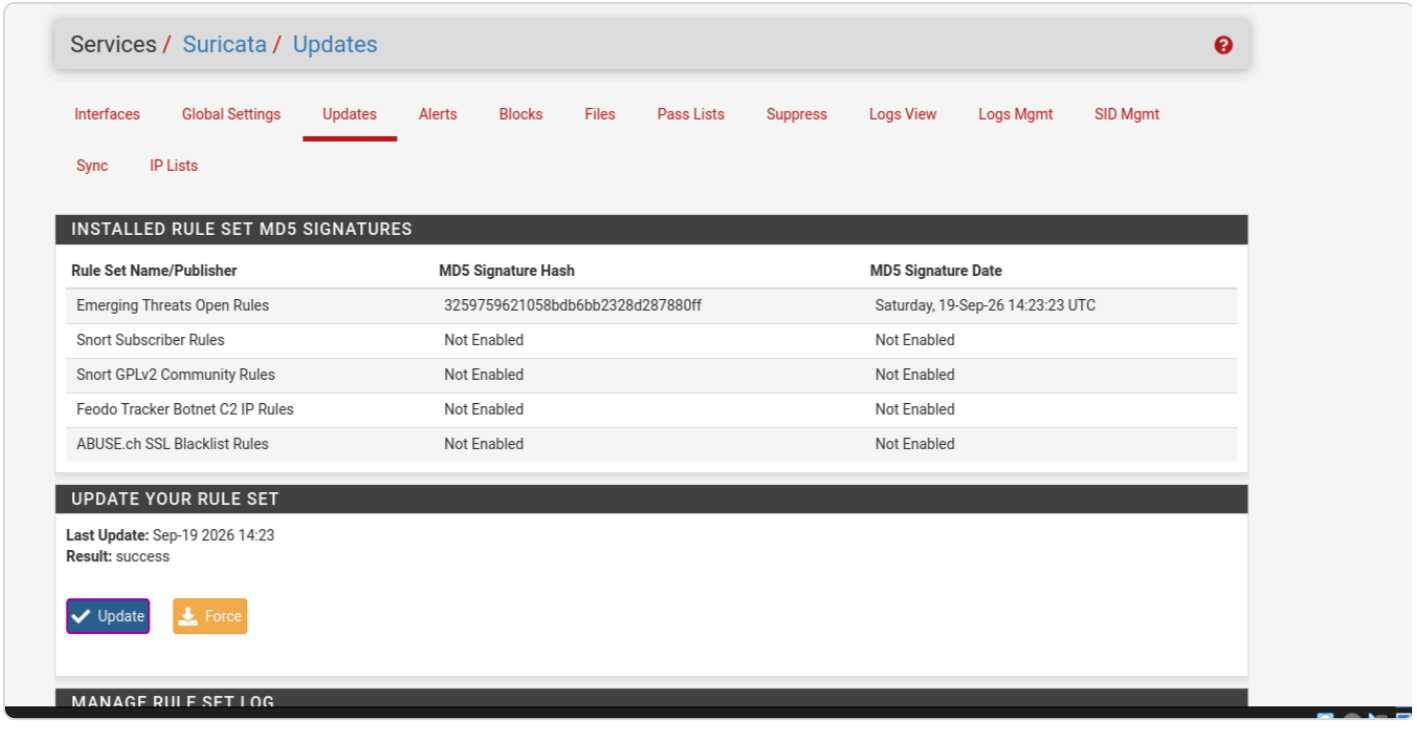



I added the **WAN** interface, the side facing Metasploitable, which is the side I want to protect:

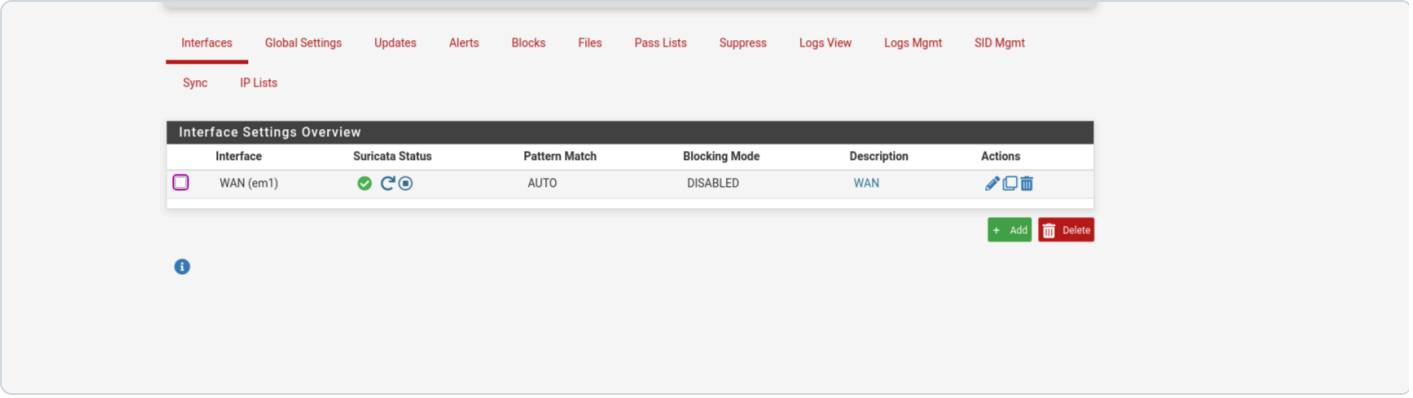



I also confirmed the **Emerging Threats Open** rule set had downloaded. The MD5 hash and timestamp prove it:




Then I started Suricata on WAN. The green tick means it is running:




**Where things stood:** Suricata installed, running, watching the WAN, with real rules loaded.

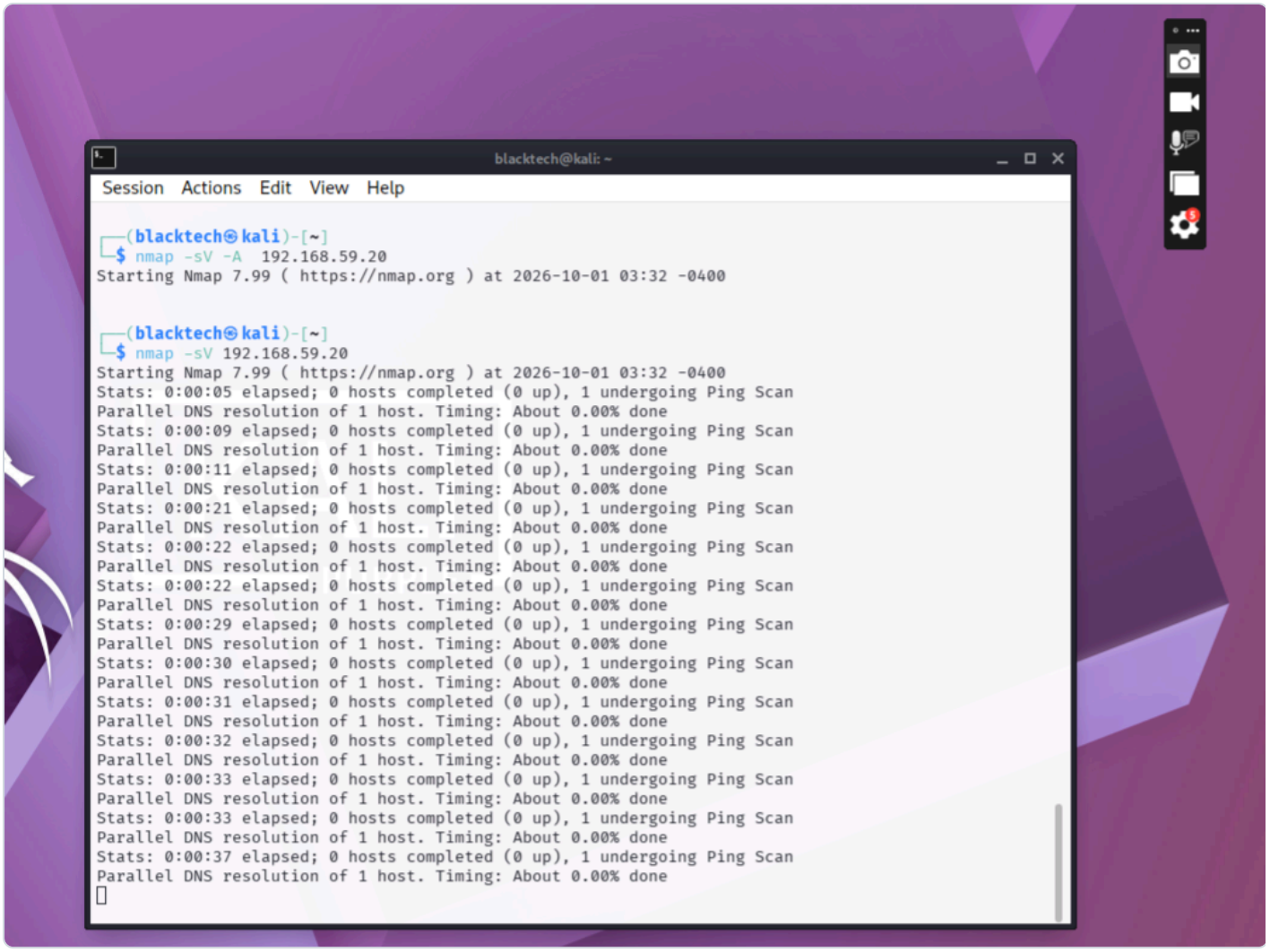
---

## Part 4: The scans that said "filtered"

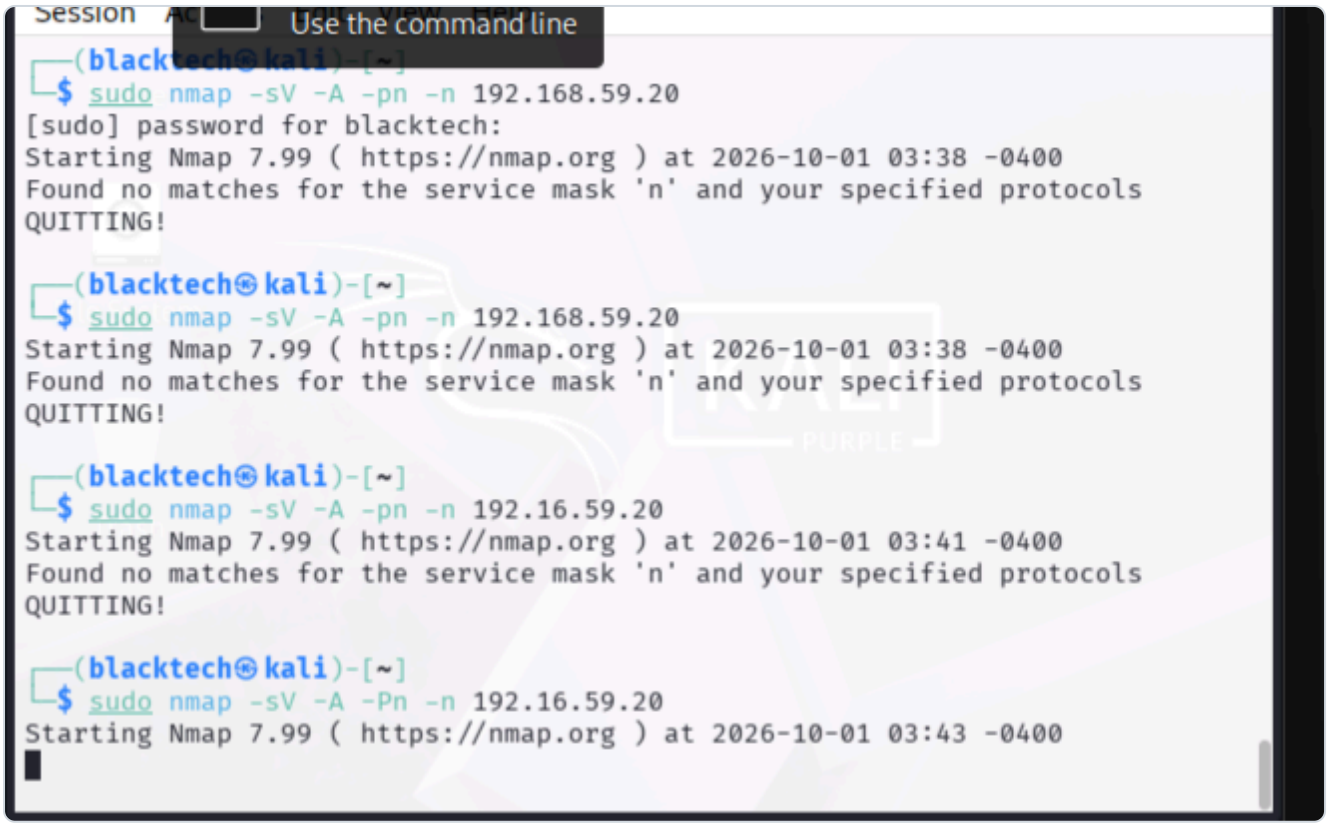
To get Suricata to alert, I needed to attack Metasploitable from Kali with Nmap. This is where the second long problem began.

### First, mistakes of my own

- The first scan hung on "Ping Scan: 0.00% done" for minutes:




- I typed `-pn` (lowercase) instead of `-Pn`, and wrong IP addresses (`192.16.59.20`, `192.168.168.59.20`):

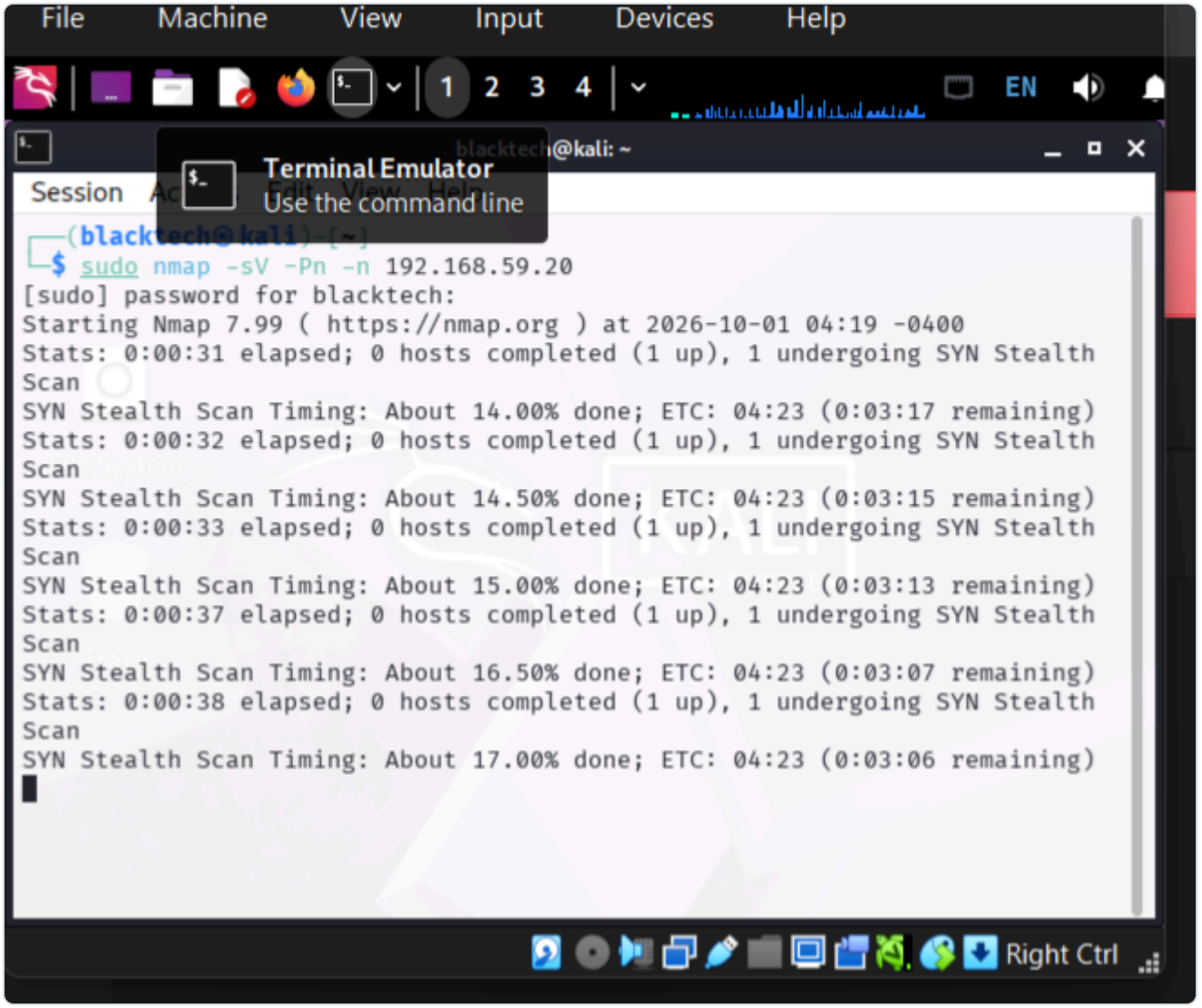


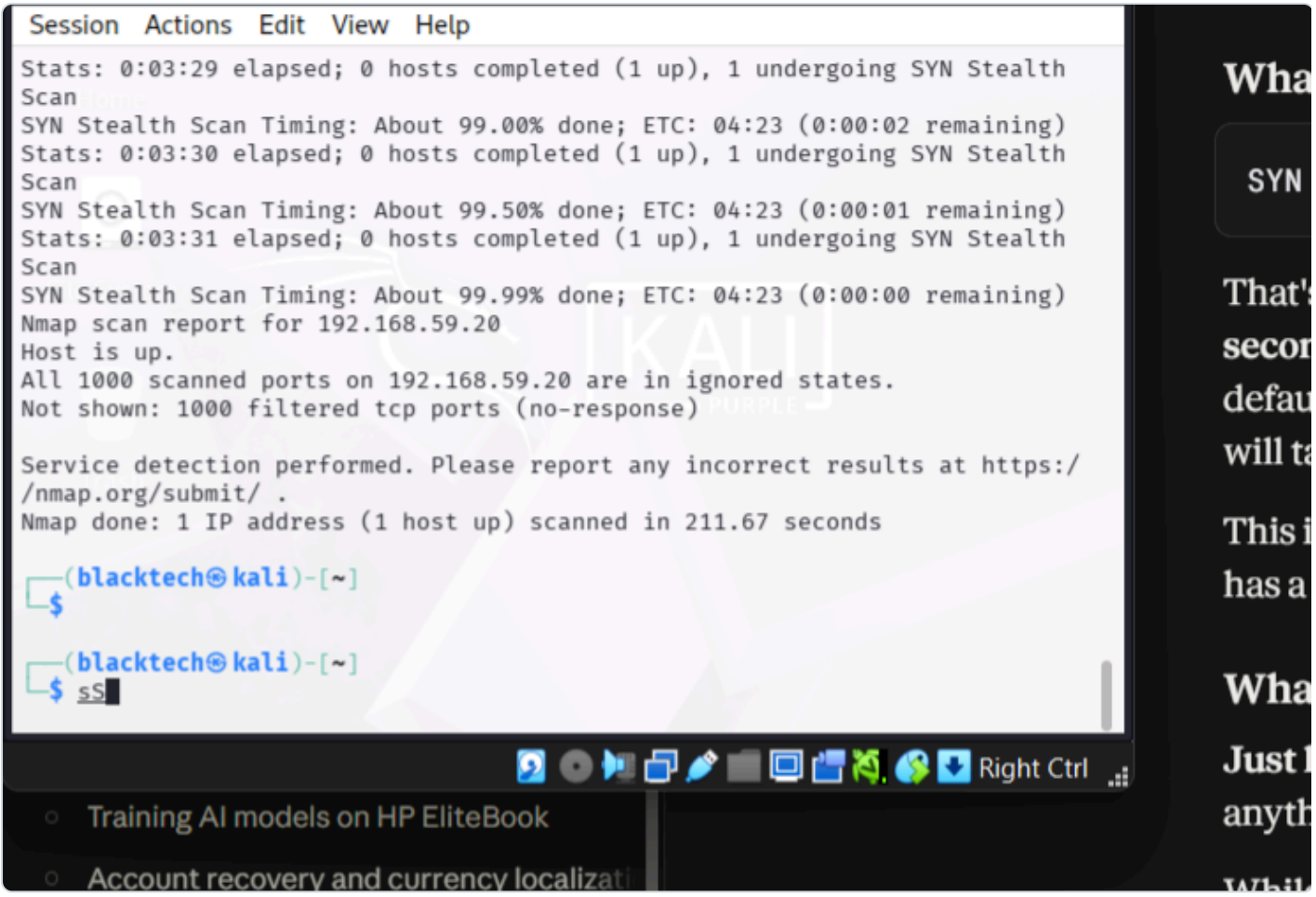

*Capital `-P` is about the "ping" check. Lowercase `-p` selects ports. Case matters.*

### The scan that finished, with no good news

A SYN scan ( `-sS` , the default for root) took **211 seconds** and reported:

> All 1000 scanned ports on 192.168.59.20 are in ignored states. Not shown: 1000 filtered tcp ports (no-response)

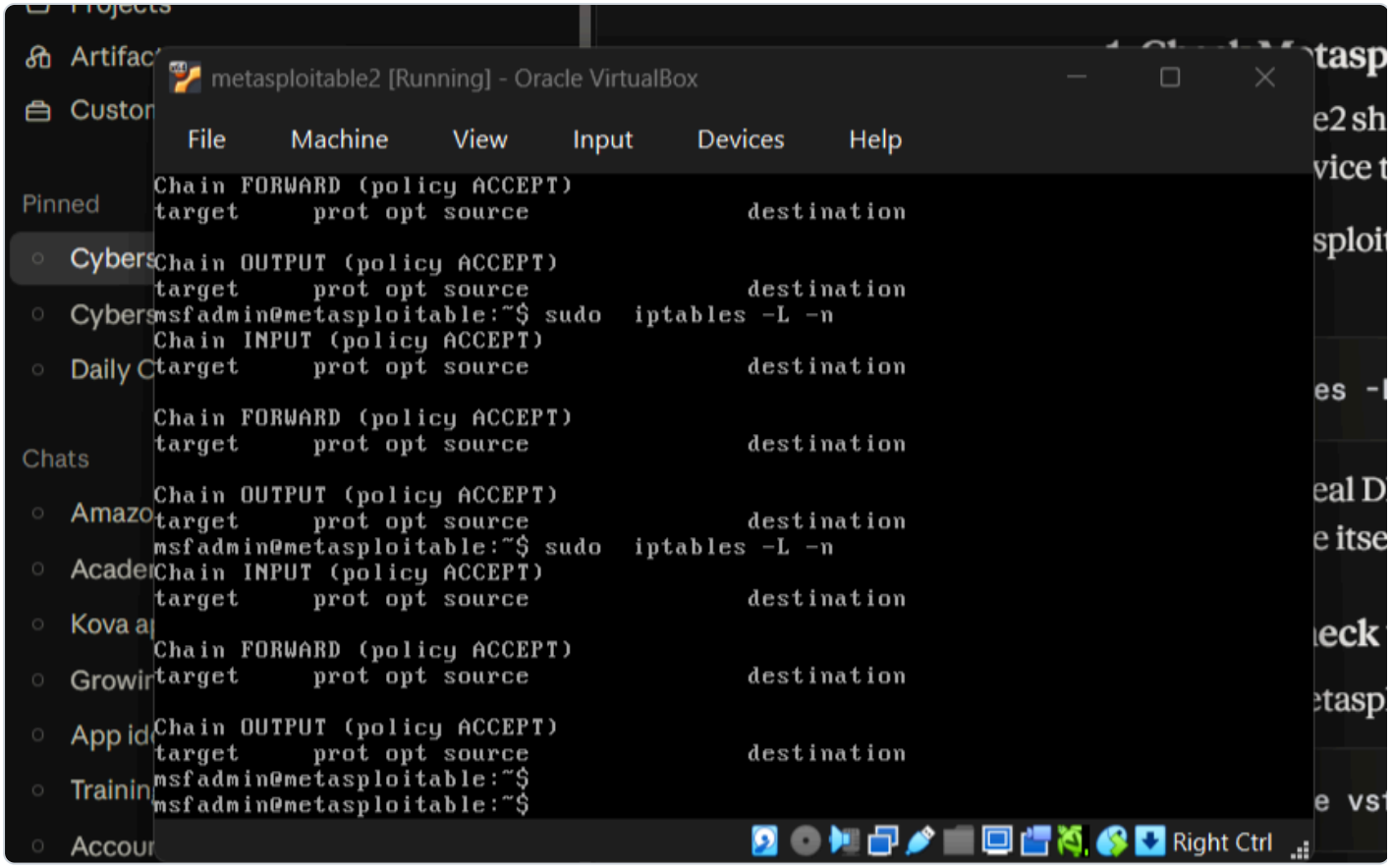

File
Machine
View
Input
Devices
Help
1 2 3 4
EN
Terminal Emulator
Use the command line
@kali: ~
Session
(blacktech㉿kali)-[~]
$ sudo nmap -sV -Pn -n 192.168.59.20
[sudo] password for blacktech:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-01 04:19 -0400
Stats: 0:00:31 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 14.00% done; ETC: 04:23 (0:03:17 remaining)
Stats: 0:00:32 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 14.50% done; ETC: 04:23 (0:03:15 remaining)
Stats: 0:00:33 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 15.00% done; ETC: 04:23 (0:03:13 remaining)
Stats: 0:00:37 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 16.50% done; ETC: 04:23 (0:03:07 remaining)
Stats: 0:00:38 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 17.00% done; ETC: 04:23 (0:03:06 remaining)
Right Ctrl

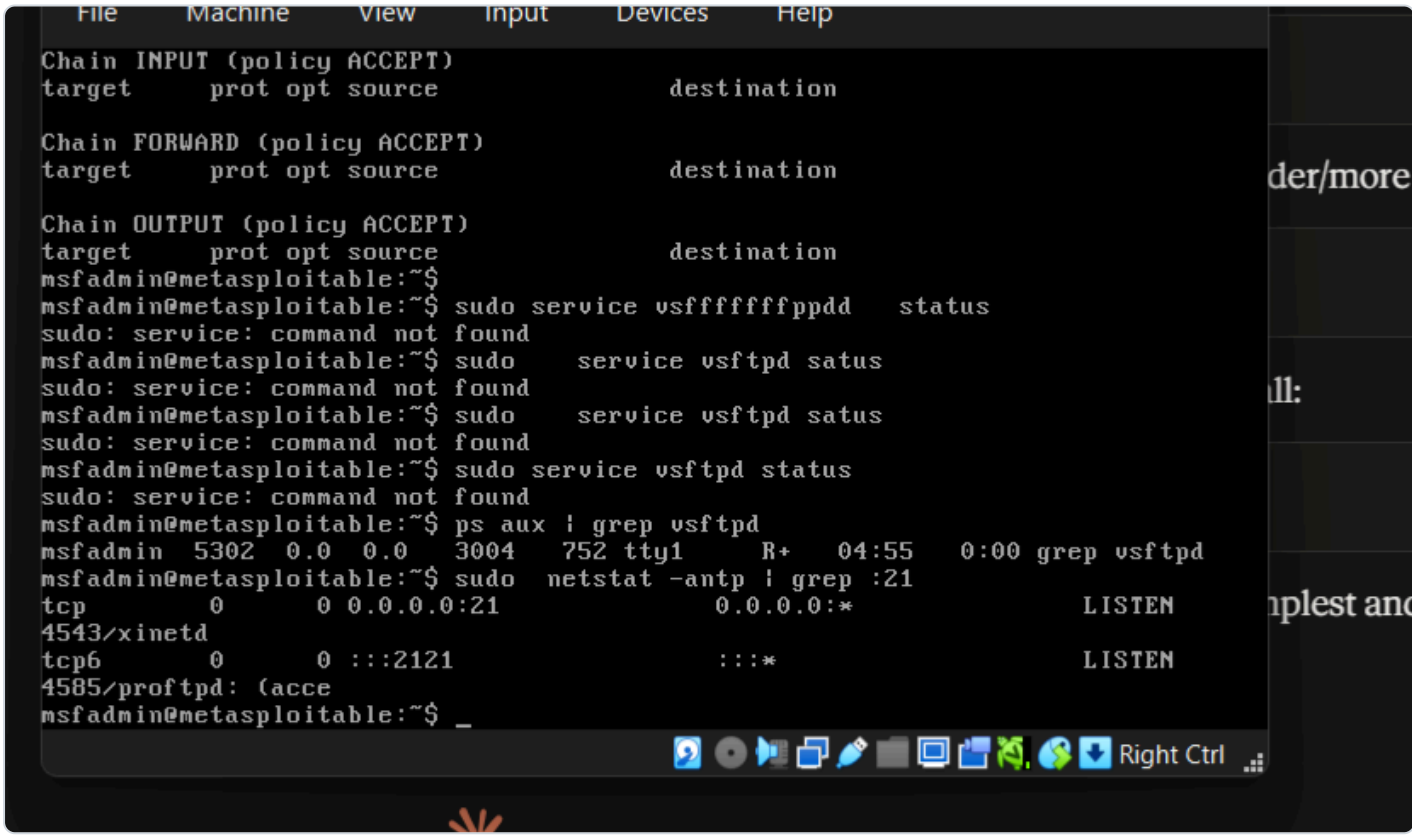


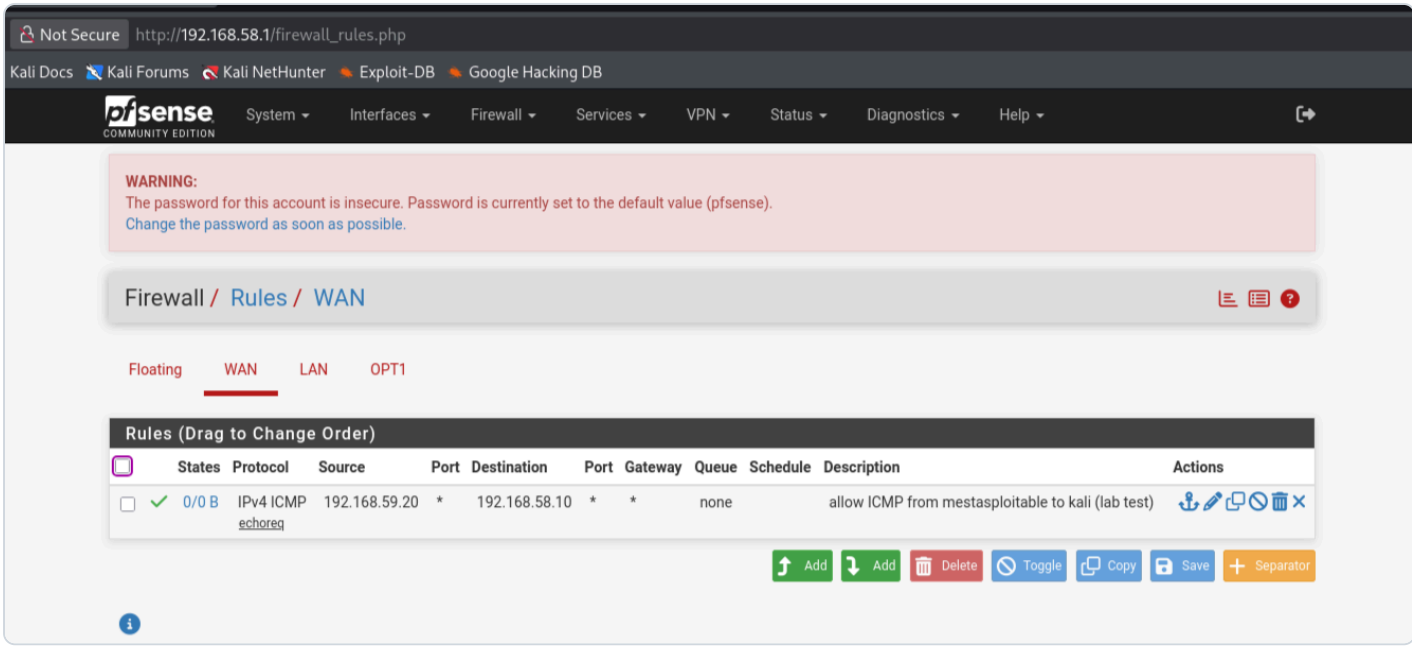

**What "filtered" means:** Nmap sent packets and got **no answer at all**. It does not say who dropped them or why. It is not the same as *closed*, where the target answers "no" immediately.

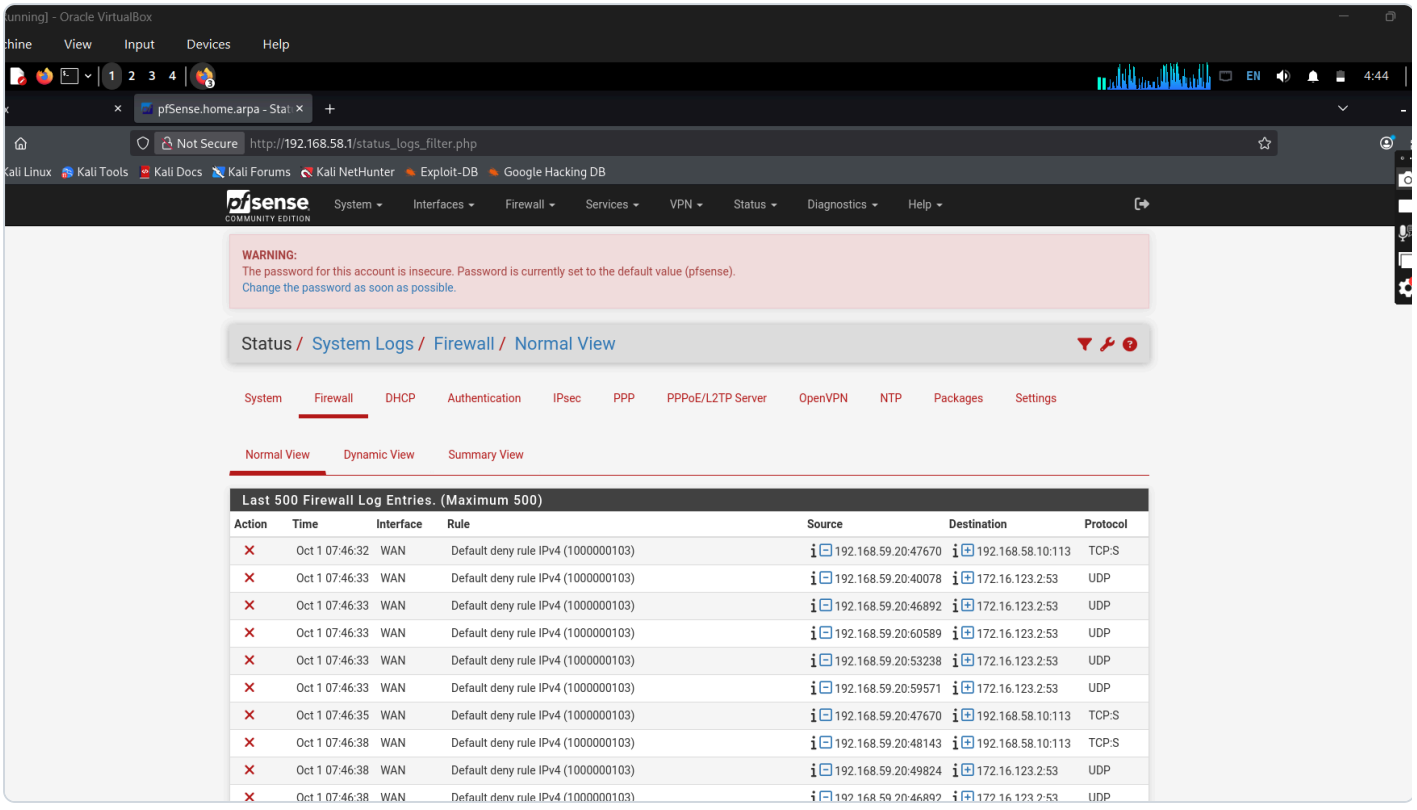
### How I checked the obvious things

| Check | Result | What it showed |
|---|---|---|
| Metasploitable's own firewall ( `sudo iptables -L -n` ) | All chains policy ACCEPT, no rules | The target is not blocking |
| Is the FTP service running? ( `sudo netstat -antp \| grep :21` ) | `0.0.0.0:21 LISTEN` handled by `xinetd` | The service is up |
| pfSense WAN rules | My ICMP allow rule showed `0/0 B` states | That rule never matched anything |
| pfSense firewall log | `Default deny` entries for traffic from 192.168.59.20 | Normal side traffic (see below) |


metasploitable2 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Chain FORWARD (policy ACCEPT)
target prot opt source destination
Chain OUTPUT (policy ACCEPT)
target prot opt source destination
msfadmin@metasploitable:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target prot opt source destination
Chain FORWARD (policy ACCEPT)
target prot opt source destination
Chain OUTPUT (policy ACCEPT)
target prot opt source destination
msfadmin@metasploitable:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target prot opt source destination
Chain FORWARD (policy ACCEPT)
target prot opt source destination
Chain OUTPUT (policy ACCEPT)
target prot opt source destination
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$
Right Ctrl


File Machine View Input Devices Help
Chain INPUT (policy ACCEPT)
target prot opt source destination
Chain FORWARD (policy ACCEPT)
target prot opt source destination
Chain OUTPUT (policy ACCEPT)
target prot opt source destination
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$ sudo service vsfffffffppdd status
sudo: service: command not found
msfadmin@metasploitable:~$ sudo service vsftpd satus
sudo: service: command not found
msfadmin@metasploitable:~$ sudo service vsftpd satus
sudo: service: command not found
msfadmin@metasploitable:~$ sudo service vsftpd status
sudo: service: command not found
msfadmin@metasploitable:~$ ps aux | grep vsftpd
msfadmin 5302 0.0 0.0 3004 752 tty1 R+ 04:55 0:00 grep vsftpd
msfadmin@metasploitable:~$ sudo netstat -antp | grep :21
tcp 0 0 0.0.0.0:21 0.0.0.0:* LISTEN
4543/xinetd
tcp6 0 0 :::2121 :::* LISTEN
4585/proftpd: (acce
msfadmin@metasploitable:~$ _
Right Ctrl


Not Secure http://192.168.58.1/firewall_rules.php
Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
pfsense COMMUNITY EDITION
System Interfaces Firewall Services VPN Status Diagnostics Help
WARNING:
The password for this account is insecure. Password is currently set to the default value (pfsense).
Change the password as soon as possible.
Firewall / Rules / WAN
Floating WAN LAN OPT1
Rules (Drag to Change Order)
States Protocol Source Port Destination Port Gateway Queue Schedule Description Actions
0/0 B IPv4 ICMP echoreq 192.168.59.20 * 192.168.58.10 * * none allow ICMP from mestasploitable to kali (lab test)
Add Add Delete Toggle Copy Save Separator

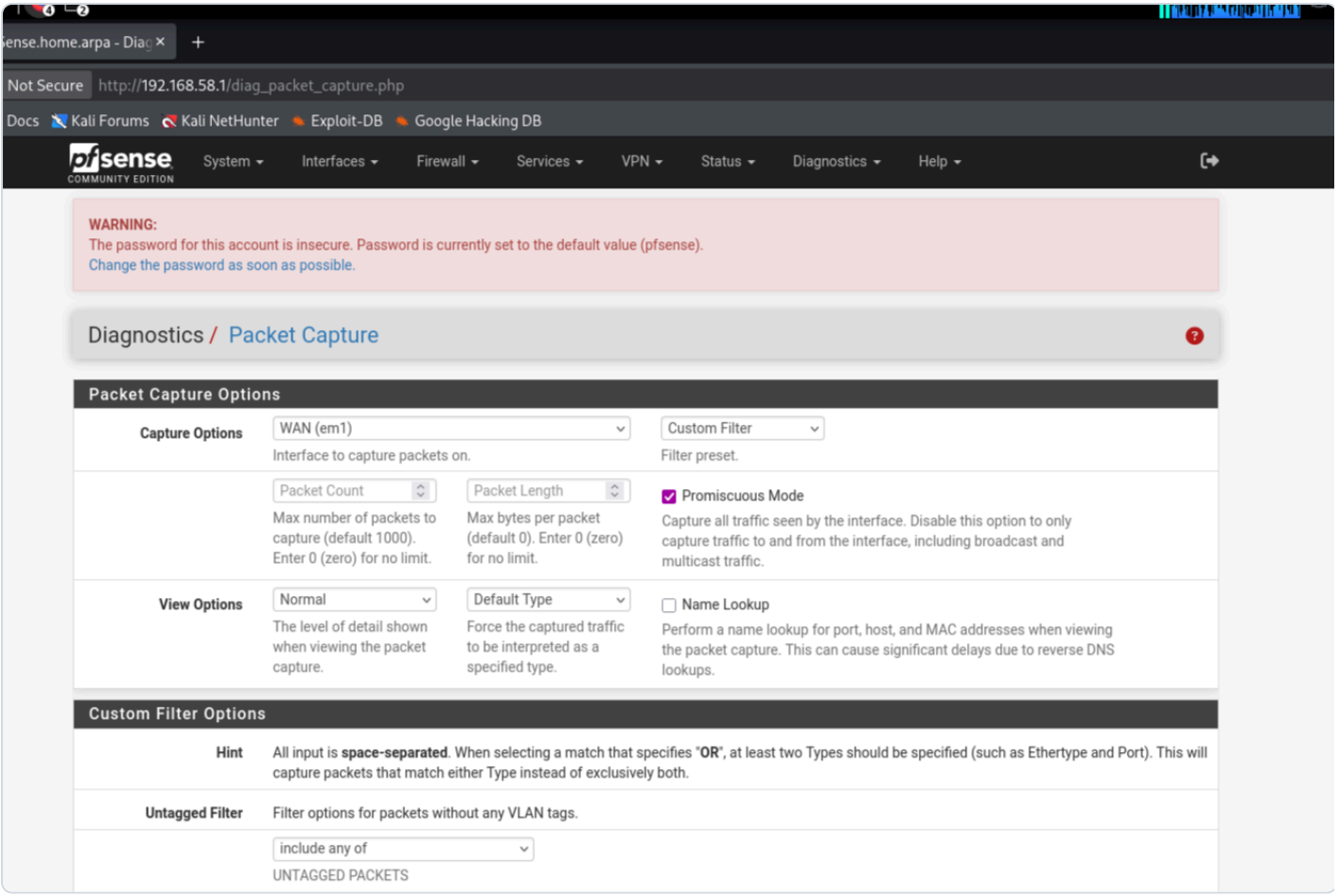

Oracle VirtualBox
View Input Devices Help
pfSense.home.arpa - Stat
Not Secure http://192.168.58.1/status_logs_filter.php
Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
pfsense COMMUNITY EDITION
System Interfaces Firewall Services VPN Status Diagnostics Help
WARNING:
The password for this account is insecure. Password is currently set to the default value (pfsense).
Change the password as soon as possible.
Status / System Logs / Firewall / Normal View
System Firewall DHCP Authentication IPsec PPP PPPoE/L2TP Server OpenVPN NTP Packages Settings
Normal View Dynamic View Summary View
Last 500 Firewall Log Entries. (Maximum 500)
Action Time Interface Rule Source Destination Protocol
Oct 1 07:46:32 WAN Default deny rule IPv4 (1000000103) 192.168.59.20:47670 192.168.58.10:113 TCP:S
Oct 1 07:46:33 WAN Default deny rule IPv4 (1000000103) 192.168.59.20:40078 172.16.123.2:53 UDP
Oct 1 07:46:33 WAN Default deny rule IPv4 (1000000103) 192.168.59.20:46892 172.16.123.2:53 UDP
Oct 1 07:46:33 WAN Default deny rule IPv4 (1000000103) 192.168.59.20:60589 172.16.123.2:53 UDP
Oct 1 07:46:33 WAN Default deny rule IPv4 (1000000103) 192.168.59.20:53238 172.16.123.2:53 UDP
Oct 1 07:46:33 WAN Default deny rule IPv4 (1000000103) 192.168.59.20:59571 172.16.123.2:53 UDP
Oct 1 07:46:35 WAN Default deny rule IPv4 (1000000103) 192.168.59.20:47670 192.168.58.10:113 TCP:S
Oct 1 07:46:38 WAN Default deny rule IPv4 (1000000103) 192.168.59.20:48143 192.168.58.10:113 TCP:S
Oct 1 07:46:38 WAN Default deny rule IPv4 (1000000103) 192.168.59.20:49824 172.16.123.2:53 UDP




About the firewall log: the entries show Metasploitable itself trying to open connections back to Kali (port 113, "ident") and to a DNS server. pfSense blocks new connections from the WAN side by default, which is correct behaviour. Those blocks are **not** the reason the scan failed.

### My theory, and why it was wrong to trust it

I guessed that pfSense's built-in "Block private networks" and "Block bogon networks" options on the WAN were dropping the traffic. I toggled the checkboxes, but I **never proved** it. I tried to confirm it with a packet capture on pfSense, but I gave up after repeated mistakes in the form (a wrong IP, for example).




I moved on to Week 6 without solving it. That open question stayed with me until the next time I picked it up.

---

## Part 5: Finding the real cause

On Nov 2 I came back and did it properly. Instead of guessing, I followed one packet along its whole trip and tested the start, the middle and the end.

### The packet's journey

1. **Kali** picks a network card from its routing table.
2. **pfSense LAN** checks its rules.
3. **pfSense** passes the packet toward the WAN.
4. **Metasploitable** receives it and replies.
5. **pfSense** matches the reply to the original request and forwards it.
6. **Kali** receives the reply and Nmap reports `open`.

"Filtered" means the packet died somewhere on that trip. The question is where.

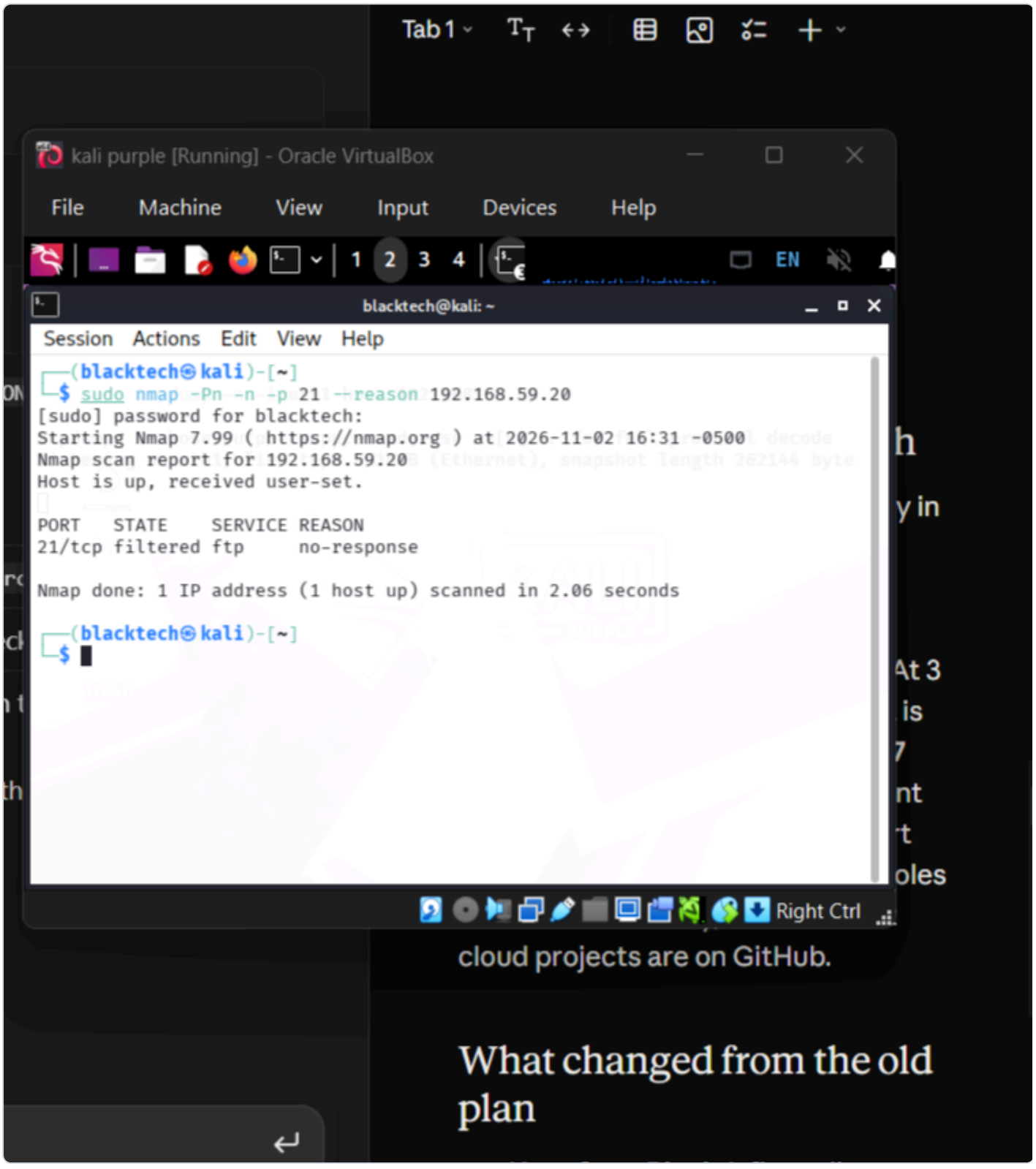
### Evidence 1: Nmap says why

```
sudo nmap -Pn -n -p 21 --reason 192.168.59.20
```

The `--reason` flag prints the exact reply. The result:

```
21/tcp filtered ftp   no-response
```

`no-response` means silent. No reset, no "prohibited" message.

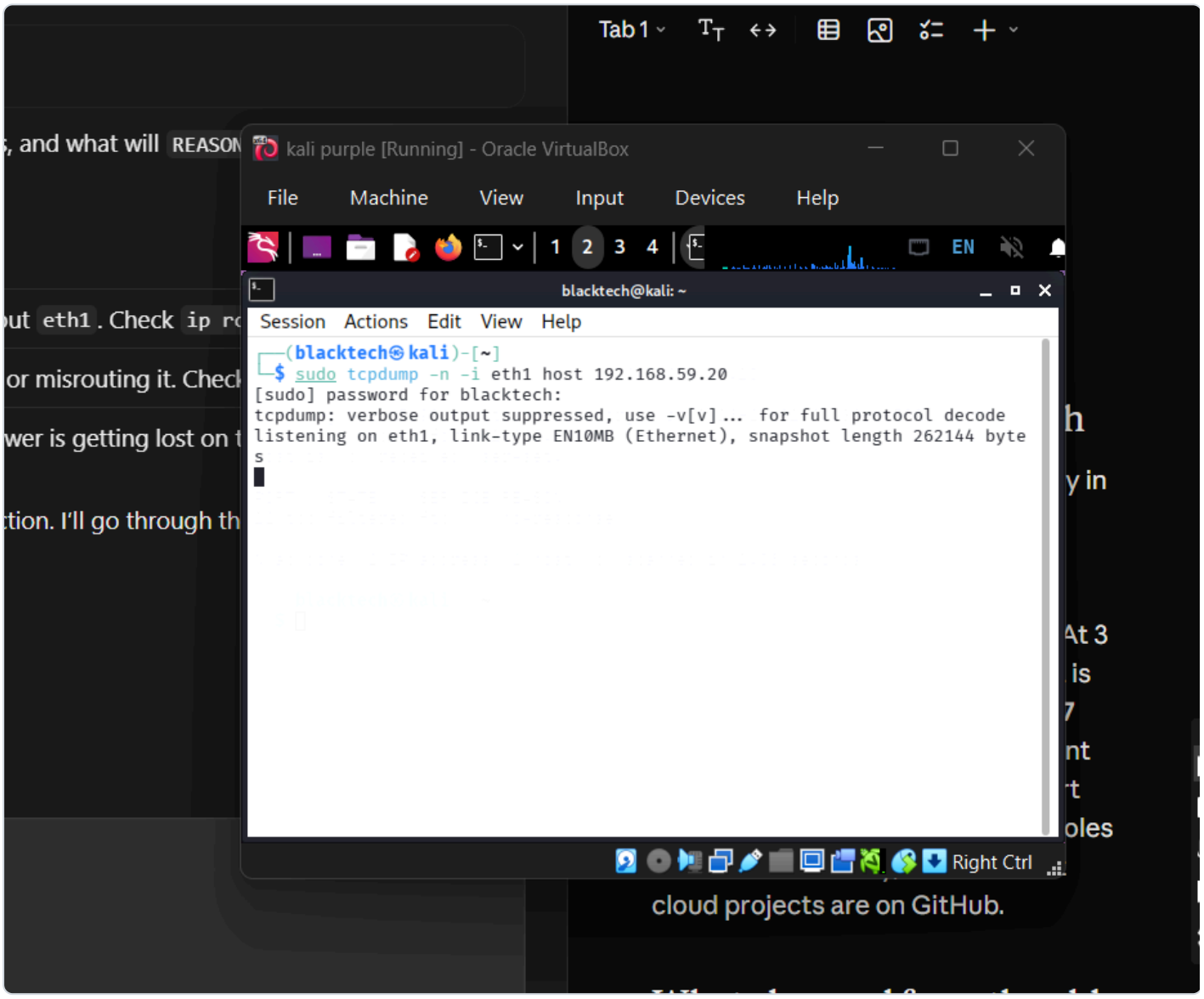



### Evidence 2: Did the packet leave Kali toward pfSense?

I listened on Kali's `eth1` (the card that faces pfSense) while scanning:

```
sudo tcpdump -n -i eth1 host 192.168.59.20
```

**Nothing appeared.** The scan packets never went out of `eth1`.




That ruled out pfSense and Metasploitable as the first suspects. The packet had not even reached them.

### Evidence 3: Which way does Kali send it?

```
ip route get 192.168.59.20
```

Result:

```
192.168.59.20 via 10.0.2.2 dev eth0 src 10.0.2.15
```

Kali was sending the packet out **`eth0`** (its internet card, via the VirtualBox NAT gateway at `10.0.2.2`), and not out `eth1` toward pfSense. Kali had no saved rule saying "to reach 192.168.59.0/24, go through

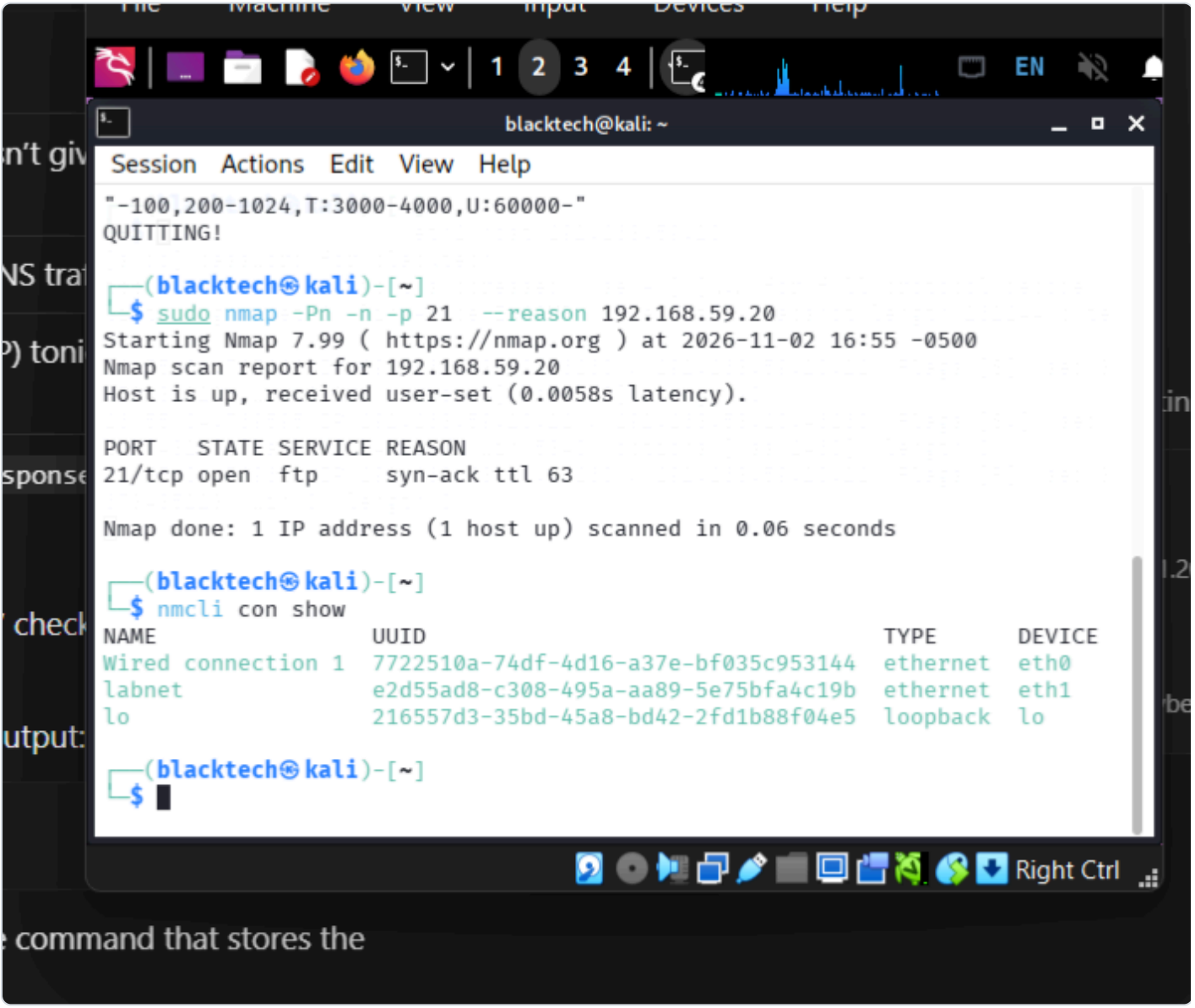
pfSense", so the packet took the default route and vanished.

### The fix

```
sudo ip route add 192.168.59.0/24 via 192.168.58.1 dev eth1
ip route get 192.168.59.20      # now: via 192.168.58.1 dev eth1
sudo nmap -Pn -n -p 21 --reason 192.168.59.20
```

The scan came back in 0.06 seconds, instead of 2 seconds of waiting:

```
21/tcp open ftp   syn-ack   ttl 63
```

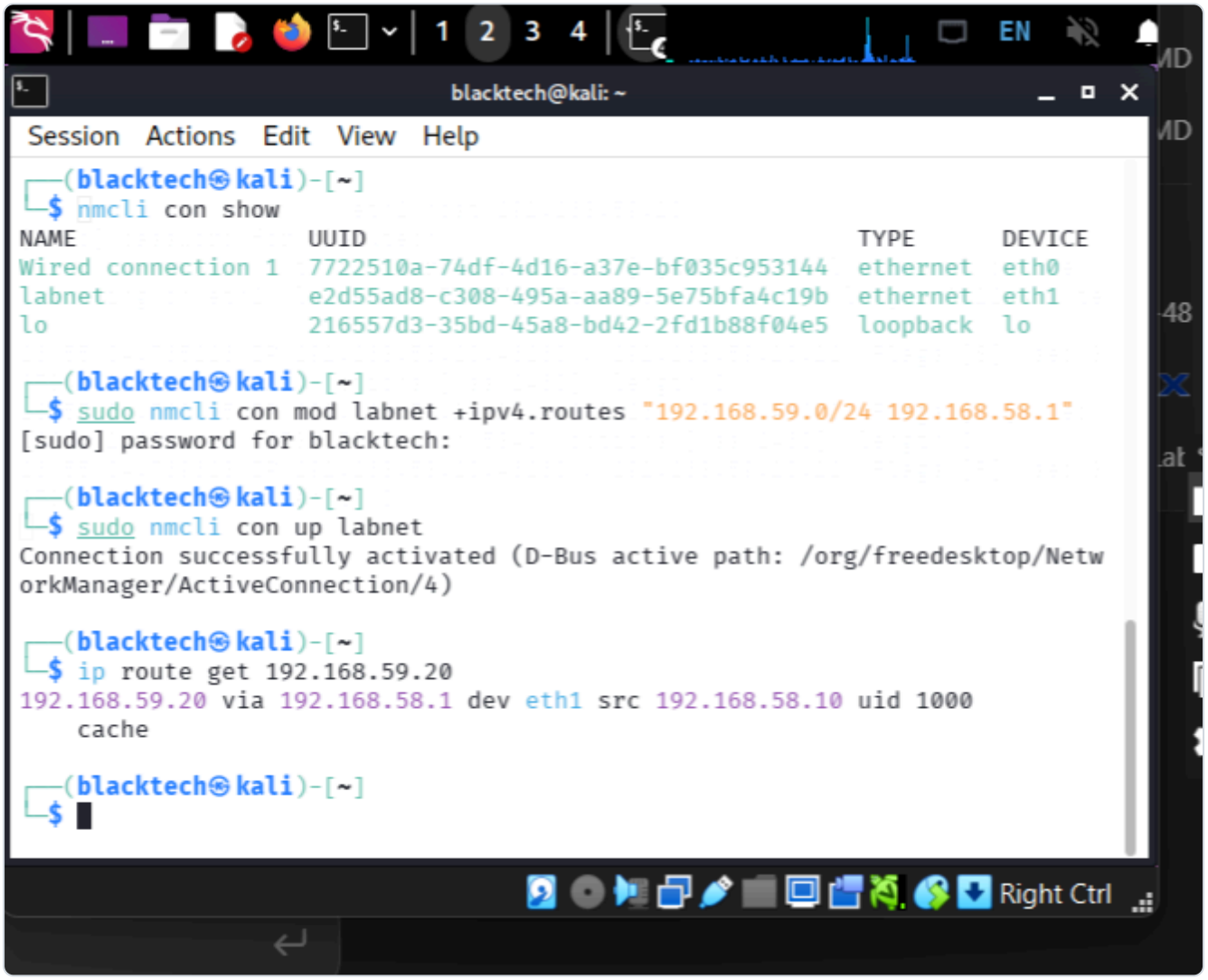



**Reading that line:** - `open` and `syn-ack`: Metasploitable answered the TCP handshake, so the service is listening. - `ttl 63`: **TTL** (Time To Live) starts at 64 on Linux and loses 1 at every router. Seeing 63 proves **exactly one router, pfSense, sat in the path**.

### Making it permanent

A route added with `ip route add` lives only in memory and disappears on reboot. I stored it in the `labnet` connection's saved settings (found with `nmcli con show`):

```
sudo nmcli con mod labnet +ipv4.routes "192.168.59.0/24 192.168.58.1"
sudo nmcli con up labnet
```




---

## Part 6: Real alerts and my own rule

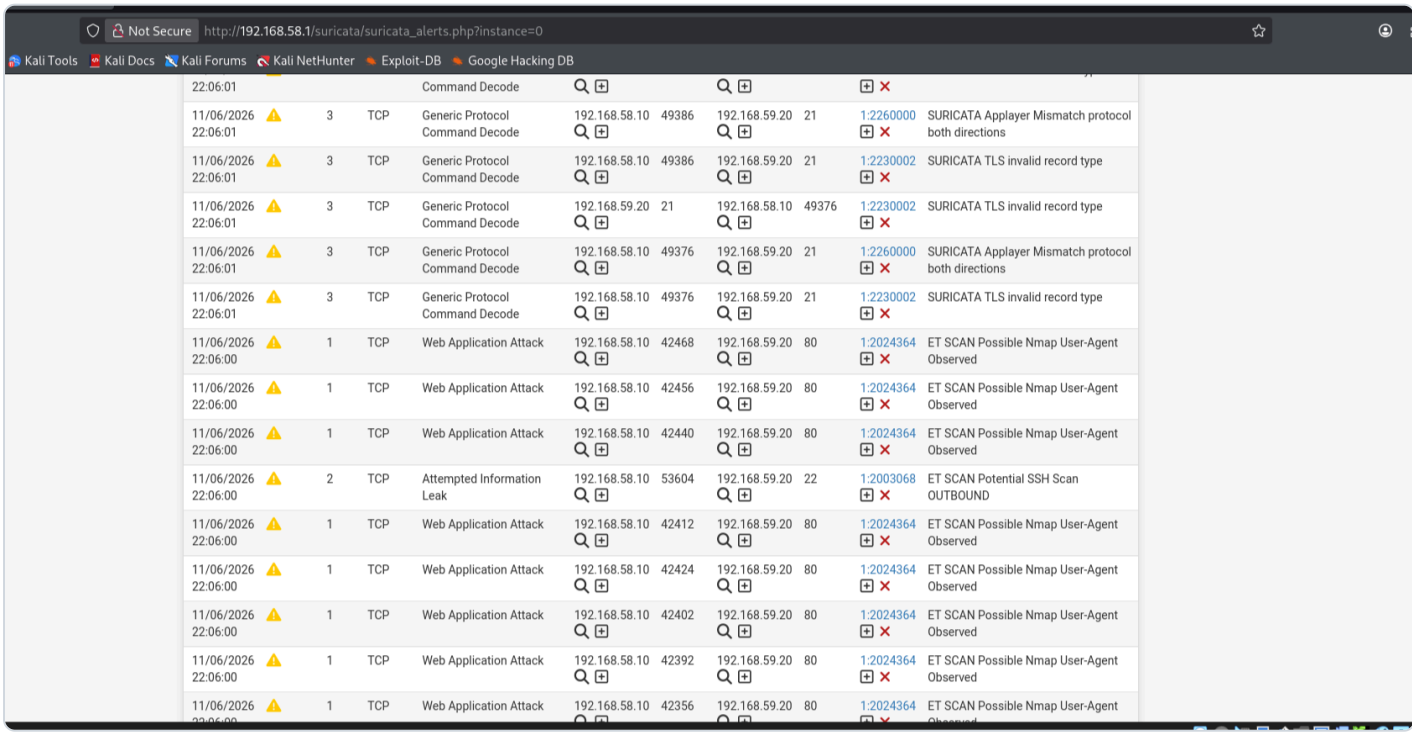
With the route fixed, I scanned Metasploitable (192.168.59.20) from Kali on Nov 6 (with the pfSense VM running; the first scan that night said "filtered" in under 10 seconds because I had forgotten to start it).

### Alerts that appeared on their own

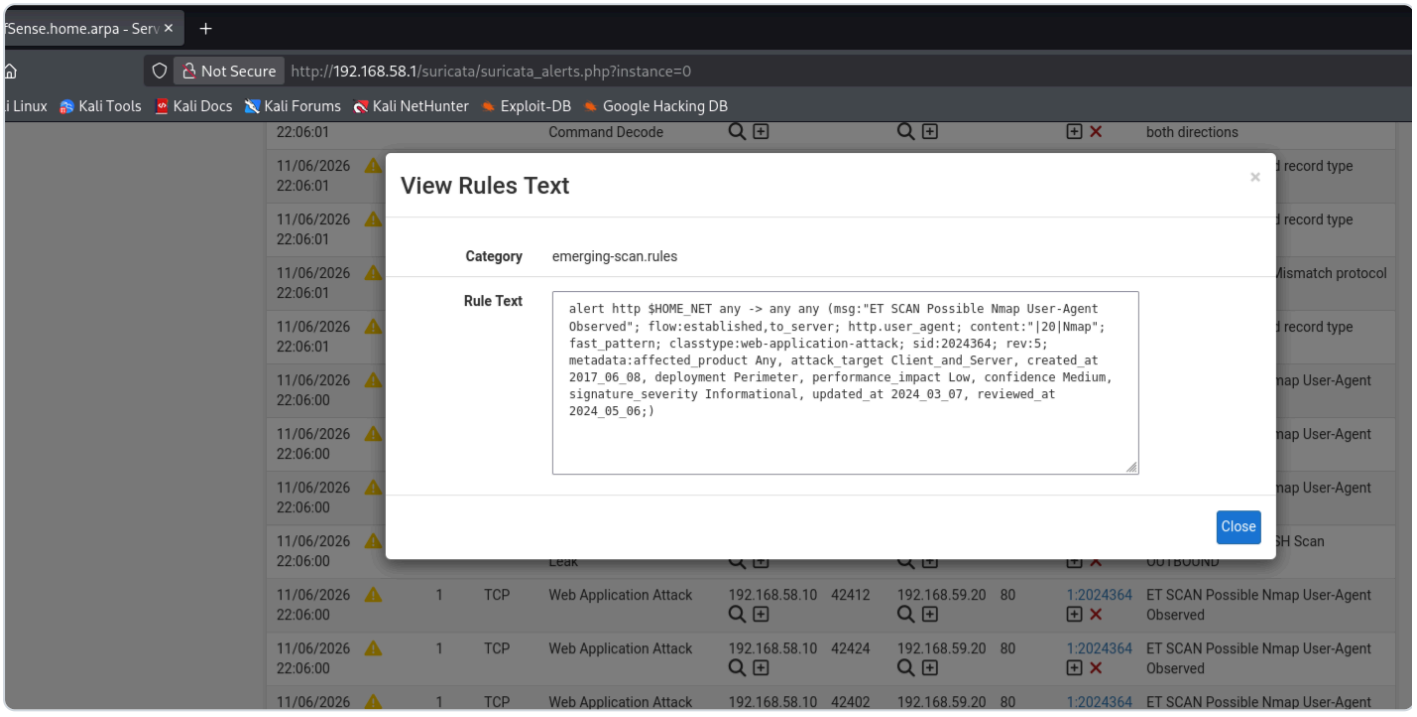
Suricata's built-in checks fired first, for example `1:2230002 TLS invalid record type` and `1:2260000 Applayer Mismatch`. These came from Nmap's `-sV` probes: Nmap sends odd data to guess the service, and Suricata notices the traffic does not look like normal FTP or SMTP.

The ET SCAN alerts only appeared **after I enabled the rule categories on the WAN interface** (Services > Suricata > WAN Categories: `emerging-scan`, `emerging-ftp`, `emerging-exploit`, `emerging-`

`attack_response`, `emerging-info`), restarted Suricata and scanned again. Having Suricata "running" is not the same as having the right rules switched on.




The rule behind `1:2024364 ET SCAN Possible Nmap User-Agent Observed` (opened with *View Rules Text*):




## How to read a rule

```
 alert http $HOME_NET any -> any any (msg:"ET SCAN Possible Nmap User-Agent Observed";
flow:established,to_server; http.user_agent; content:"|20|Nmap"; ...; classtype:web-
```

```
application-attack; sid:2024364; rev:5;)
```

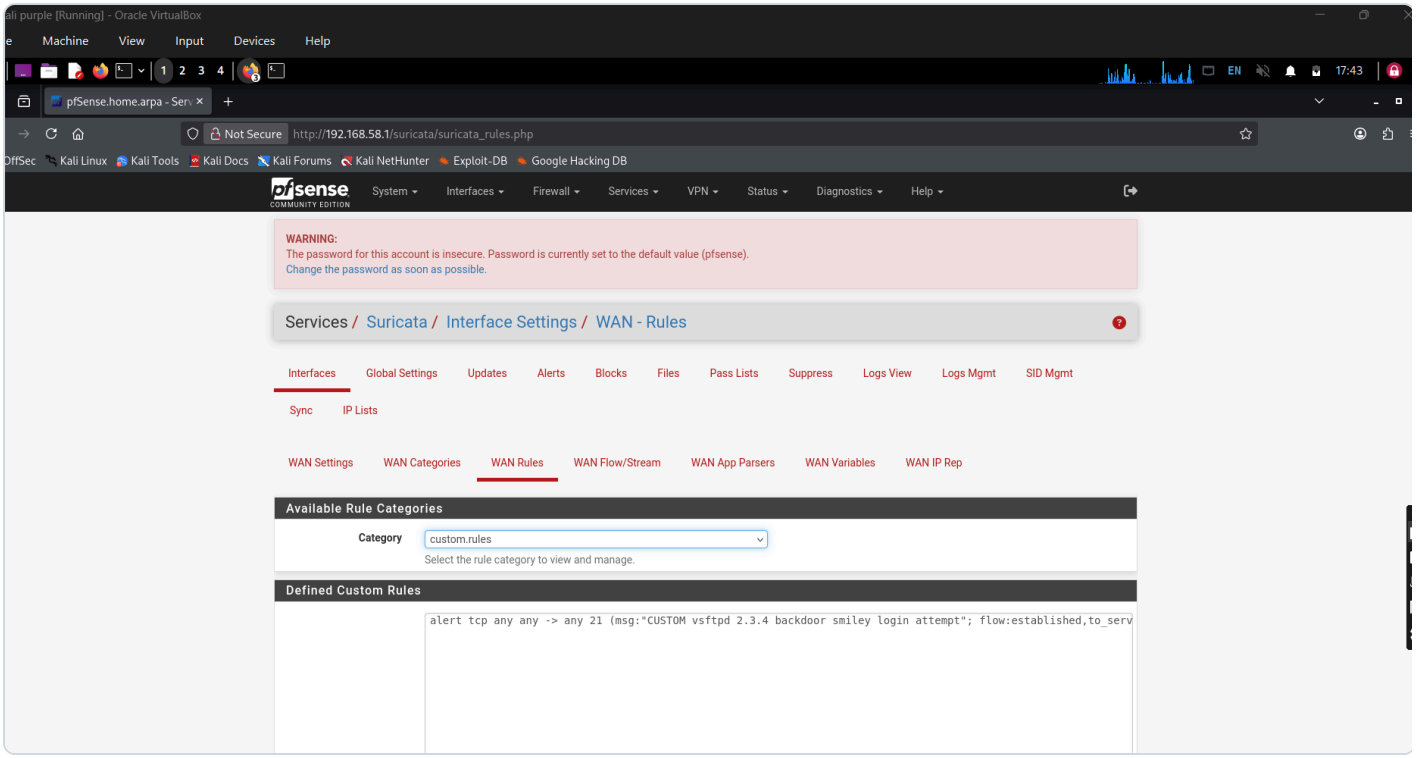
| Part | Meaning |
|---|---|
| `alert` | The action: raise an alert (my Suricata is in IDS mode, it does not block) |
| `http` | The protocol it understands |
| `$HOME_NET any -> any any` | Source address and port, direction, destination address and port |
| `msg` | The text shown in the Alerts tab |
| `flow:established,to_server` | Only after the connection is set up, only traffic going to the server |
| `http.user_agent; content:"Nmap"` | Look inside the HTTP User-Agent header for the word Nmap |
| `classtype`, `sid`, `rev` | Category, unique rule number, version |

Why the source is `$HOME_NET`: Suricata treats Kali (192.168.58.10) as "inside" my network, so a scan *from* Kali matches a rule written for traffic *from* the home network. If `$HOME_NET` is wrong, rules like this silently never fire.

### My own rule

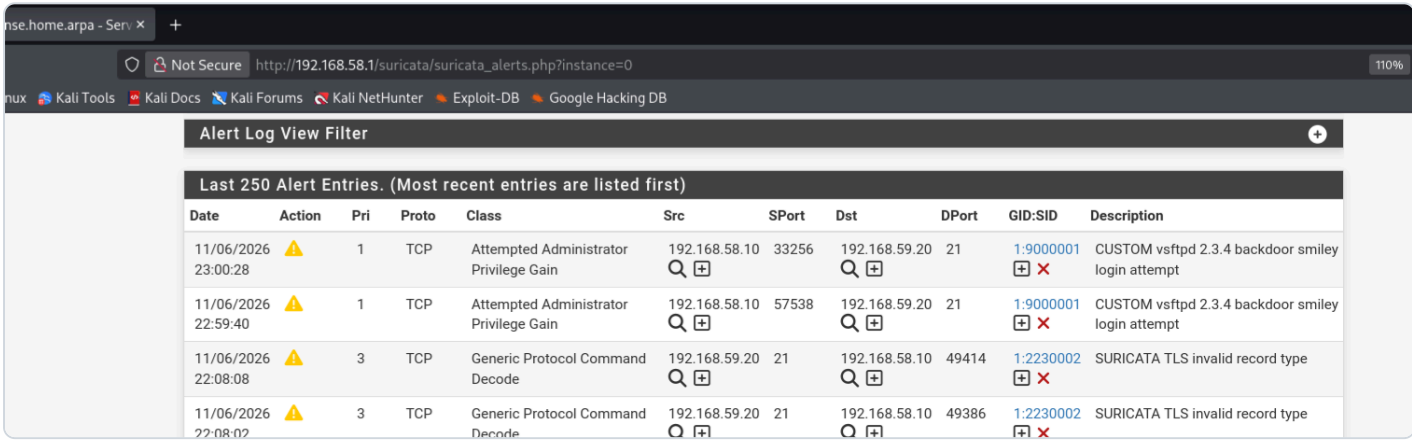
The vsftpd 2.3.4 version in Metasploitable has a known backdoor: a username that ends in a smiley `:)` makes the server open a shell on port 6200. I wrote a custom rule in `custom.rules` (Services > Suricata > WAN Rules) that alerts on an FTP login containing that smiley. Custom rule numbers start at `9000000` so they never clash with the ET rules. The rule starts:

```
alert tcp any any -> any 21 (msg:"CUSTOM vsftpd 2.3.4 backdoor smiley login attempt";
flow:established,to_server; ...  sid:9000001; ...)
```




*(Add your complete rule line here before publishing, copied from `custom.rules`.)*

After saving, restarting Suricata and sending the smiley login from Kali, **my rule fired twice** (`1:9000001`, priority 1):




**What the rule gets wrong:** it only looks for the smiley text, so a harmless login that happens to contain `:)` also alerts (a false positive), and it does not prove the backdoor opened. A better version would also alert on a connection to port 6200, the shell the backdoor creates.

---

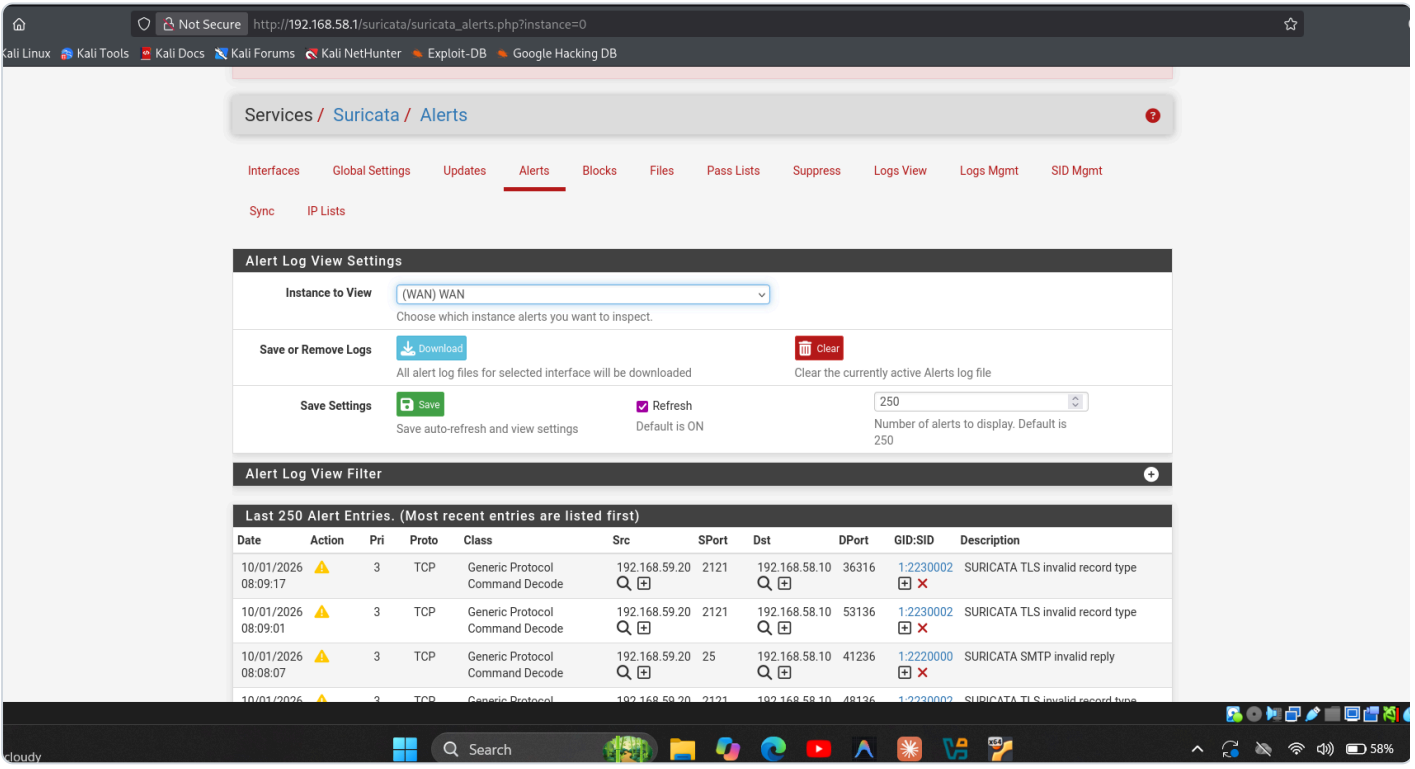
## What I learned

1. **Do not blame the obvious suspect without evidence.** I blamed the firewall for weeks. The packet had never reached it.
2. **"Filtered" is a symptom, not a diagnosis.** It means "no answer". Finding out why needs evidence along the path.
3. **Bisect.** Test the start, middle and end of the path to find which half the packet dies in.

4. **Measure with the right tool.** `tcpdump` on the sending side answered in one minute what weeks of theories could not.
5. **`ip route get` is a truth-teller.** It shows what Linux will actually do with a packet.
6. **"Host is up" under `-Pn` means nothing.** With `-Pn`, Nmap prints it without checking.
7. **A failing install is often about what is underneath** (old OS, old library), not the thing being installed.
8. **A stale banner is not proof.** Test the real feature.

### Honest caveat

The Nov 2 cause is fully proven by the three pieces of evidence above. The Oct 1 cause is **not** proven. I did not check Kali's route that day, and the old Oct 1 alerts show Metasploitable *replying* from ports 25 and 2121, so on Oct 1 at least some traffic did reach it. I cannot say it was the same problem. What I can say is that today the path works end to end and the fix survives (the reboot re-check is still owed).




---

## Cheat sheet

| Command | What it does |
|---|---|
| `ip route get <IP>` | Shows the card and gateway Linux will use for that IP |
| `ip route` | Prints the whole routing table |
| `sudo tcpdump -n -i eth1 host <IP>` | Prints packets on a card for one IP |
| `sudo nmap -Pn -n -p 21 --reason <IP>` | Scans one port and says why it got its result |
| `sudo ip route add <net> via <gw> dev <if>` | Adds a temporary route |

| Command | What it does |
|---|---|
| `nmcli con show` | Lists saved network connections |
| `sudo nmcli con mod <name> +ipv4.routes "<net> <gw>"` | Saves a route permanently |
| `IGNORE_OSVERSION=yes pkg-static update -f` | Forces a fresh pfSense package catalog |
| `pkg-static delete -y <pkg>` | Removes a package without asking |

| Nmap flag | Meaning |
|---|---|
| `-Pn` | Skip the "is anyone home?" check |
| `-n` | No DNS lookups |
| `-p 21` | Scan port 21 only |
| `--reason` | Show why each port got its verdict |
| `-sV` | Detect service versions |

| Nmap result | Meaning |
|---|---|
| `open` | Something answered the handshake |
| `closed` | The host answered "no" |
| `filtered` | No answer at all, cause unknown |

---

## Still open

- Add the complete custom rule line to Part 6.
- Improve the custom rule to also watch for port 6200.
- The Week 4 ICMP allow rule (WAN) still shows `0/0 B` states. Re-test it now that the route works.
- Re-check the saved route after the next Kali reboot.
- Change the default pfSense admin password.